Software-as-a-Graph: Heterogeneous Graph Learning for Pre-Deployment Dependability Analysis of Complex Distributed Systems

Abstract

Modern distributed software systems—spanning event-driven publish–subscribe meshes, cloud-native microservices, and distributed AI serving infrastructures—decouple components to achieve elasticity and throughput. However, this decoupling obscures how cascading failures and performance degradations propagate across message brokers, asynchronous topics, shared libraries, and host nodes. Identifying systemically critical components prior to deployment is difficult because operational telemetry does not yet exist, while static code analysis cannot capture distributed communication topologies. Uncontained cascades in production trigger compute-intensive restart loops, failover storms, and severe latency tail amplification, resulting in costly downtime, excessive energy consumption, and infrastructure over-provisioning. Furthermore, applying modern AI to performance and dependability engineering often produces black boxes that lack the interpretability required for actionable remediation.

This paper presents **Software-as-a-Graph (SaG)**, an AI-driven static system analysis framework for pre-deployment dependability, performance, and computational sustainability assessment. SaG models distributed architectures as typed multigraphs over Applications, Brokers, Topics, Execution Nodes, and Shared Libraries. It integrates two decoupled pathways: (1) a **predictive pathway** employing a relation-specific **Heterogeneous Graph Transformer (HGT)** with continuous-categorical Quality-of-Service (QoS) edge encodings to forecast cascading failure blast radii, rank critical components, and score per-relationship criticality directly from Architecture-as-Code manifests; and (2) an **interpretable explanation layer** grounded in ISO/IEC 25010 and 25019 that resolves the black-box opacity of deep learning by diagnosing whether fragility stems from single points of failure, cascade hubs, or high-coupling maintenance bottlenecks, thereby prescribing targeted architectural remediations.

We evaluate SaG across 1,770 components from seven synthetic scenarios and three authentic production systems (Autoware.universe ROS 2, GCP Online Boutique, and Train-Ticket) against independent discrete-event cascade simulators under a strict input–label independence guarantee:

- **Typing enables transfer:** On unseen architectures, the typed model achieves $\rho = 0.608$, outperforming its homogeneous counterpart by $+0.353$ ($\rho = 0.439$ vs. 0.086) across all eight leave-one-scenario-out folds ($p = 0.008$). QoS edge encoding contributes an additional $+0.169$ out-of-distribution ($p = 0.016$).
- **Accurate critical-set detection at CI/CD speed:** The predictor identifies the critical top-K component set with $F_1@K = 0.414$. Inference requires only 44 ms on a 2,000-component system, replacing hours of energy-intensive multi-seed dynamic simulation and enabling green, per-commit architectural gating.
- **Transparent empirical boundary:** Compared to a training-free QoS-weighted structural baseline, the ranking advantage is $+0.037$ and is not statistically significant ($p = 0.64$). Graph learning earns its cost through cross-architecture generalization, typed relational attention, and multi-task attribution rather than raw ranking accuracy alone.
- **Real-world generalization:** SaG demonstrates zero-shot transfer to production cyber-physical and cloud-native systems ($\rho = 0.688$ to 0.778), capturing up to 100% of failure-impactful services.

By establishing continuous architectural analysis in CI/CD prior to deployment, SaG bridges the Architecture–Code Gap and advances dependable, high-performance, and computationally sustainable modern software systems.

Keywords: Graph representation learning, Heterogeneous graph neural networks, Distributed systems dependability, Cascading failures, Static system analysis, Explainable AI, Software performance and sustainability, Sustainable software engineering, CI/CD quality gates

1. Introduction

1.1. Motivation

Modern large-scale distributed software systems increasingly rely on asynchronous, event-driven, and publish–subscribe (pub-sub) architectures. Spanning autonomous vehicles (ROS 2 [1]), high-throughput enterprise event meshes (Apache Kafka [2]), cyber-physical systems (DDS [3]), IoT networks (MQTT [4]), cloud-native microservice meshes, and modern distributed AI/LLM serving backbones, pub-sub decouples producers and consumers in space, time, and synchronization [5]. Components communicate indirectly by exchanging messages through intermediate topics and brokers without maintaining direct static references to one another. Furthermore, modern middleware standards allow software engineers to declare deployment-time Quality-of-Service (QoS) contracts—such as transport reliability, durability, message priorities, and delivery deadlines—to govern how data flows under stress.

While this architectural decoupling confers elastic scalability and operational agility, it introduces a severe **visibility barrier** for system performance, reliability, and computational sustainability:

- **Indirect Failure and Degradation Pathways:** In traditional synchronous architectures (e.g., RESTful HTTP or RPC calls), component interactions follow explicit caller–callee invocation paths. In asynchronous pub-sub and event-driven meshes, publishers and subscribers have no direct static references. Cascading failures, queue head-of-line blocking, and backpressure propagate across hidden logical pathways spanning brokers, shared topics, colocated execution nodes, and shared software libraries.

- **Distinct Degradation Mechanisms:** Disturbances in complex distributed systems do not propagate in a uniform manner. They manifest either as *sequential cascades* (e.g., a slow subscriber causing broker queue saturation and upstream backpressure) or as *simultaneous blast radii* (e.g., a shared runtime library crash, memory exhaustion, or physical host outage instantly disabling multiple colocated services simultaneously). Conventional architectural diagrams and static call graphs fail to model these multi-layer interactions.

Addressing these architectural vulnerabilities is most effective and cost-efficient **prior to deployment**, during architectural design and Continuous Integration / Continuous Delivery (CI/CD). However, at design and build time, **no runtime telemetry, distributed tracing, or operational logs exist**. Consequently, software architects, performance engineers, and Site Reliability Engineers (SREs) face two fundamental questions without runtime operational data:

1. *Which components, message topics, and communication links are systemically critical to system dependability and performance?*
2. *Why are they critical, and what specific architectural fix (such as replicating a message broker, decoupling an over-subscribed topic, or sandboxing a shared library) will most effectively eliminate that risk?*

Resolving this challenge is equally paramount for **system performance and computational sustainability**. When cascading failures reach production environments, they trigger vicious cycles of compute-intensive restart loops, retry storms with exponential backoff, cluster-wide failover thrashing, and severe tail-latency amplification. These pathologies consume substantial CPU cycles, memory buffers, and network bandwidth without completing legitimate user transactions, directly translating into inflated cloud infrastructure costs, excessive energy consumption, and heightened environmental impact. Proactive, design-time architectural hardening eliminates these systemic defects before software is provisioned, preserving computational energy and protecting operational budgets.

1.2. Problem Statement: The Architecture–Code Gap and the Black-Box AI Challenge

We formulate pre-deployment dependability and performance analysis around a primary predictive task and an accompanying explanation layer that makes its output actionable:

1. **Failure-Impact Forecasting (Predictive Pathway) — the primary task.** Forecasting the dynamic, global cascading failure blast radius and ranking the systemically critical component set by training a data-driven, non-linear model over learned topological representations. No static aggregate of centrality scores can supply this: cascade reach is multi-hop and relation-dependent, and a component's blast radius depends on *which* kind of edge carries the failure outward. It is also the only task here with an independent ground truth, and the only one we evaluate as a ranking model.
2. **Explainable Criticality Attribution (Explanation Layer) — what a rank alone cannot say.** A ranked shortlist tells an engineer *where* to act but not *what to do*. We therefore pair the predictor with an interpretable structural quality profile grounded in ISO/IEC 25010 [6] and ISO/IEC 25019 [7], which diagnoses the *qualitative nature* of a flagged component's vulnerability—distinguishing, say, a single point of failure from a high-coupling maintenance bottleneck—and therefore which remediation applies. It is not a ranking model, and we do not evaluate it as one.

This separation is architectural rather than merely presentational: both pathways operate on the same graph but share no parameters, and neither is trained on the other's output. The coupling term that could connect them is disabled by default and reported only as an ablation (Section 4.2). Maintaining this distinction enables the identification of components that are structurally central yet operationally low-impact—a diagnosis unattainable by either pathway in isolation.

Existing software engineering approaches fail to bridge what we define as the **"Architecture–Code Gap"**: *a distributed system can have pristine, 100% bug-free source code within each individual service, yet remain fragile to catastrophic global outages and tail-latency explosions due to hidden architectural single points of failure (SPOFs) or mismatched middleware Quality-of-Service (QoS) contracts.* Three prevailing paradigms leave this gap unaddressed:

- **Static Code Analysis (SCA):** Tools like SonarQube inspect source code complexity [8], modularity, and cohesion [9, 10] within single services. However, SCA cannot observe the broader distributed network, message queues, or cross-host failure propagation.
- **Runtime Chaos Engineering:** Techniques like Chaos Monkey [11] and distributed tracing inject real faults into running staging or production environments. While effective at validating live systems, they require fully deployed infrastructure, carry operational risks, incur significant computational and energy costs through repeated test executions, and arrive too late to guide initial architectural design.
- **Homogeneous Graph Centrality Metrics:** Standard network metrics (betweenness, PageRank, degree) [12, 13, 14, 15] flatten systems into simple, unweighted graphs. They treat all connections identically, failing to distinguish between an asynchronous message topic, a shared library, and an execution host.

Moreover, while machine learning has demonstrated remarkable success across software engineering, modern AI approaches applied to system dependability often function as **uninterpretable black boxes**. Deep neural models frequently output risk scores or latent embeddings without offering transparent, actionable rationales for their predictions. In mission-critical software engineering, a black-box risk score is insufficient: developers and SREs cannot refactor code or reconfigure infrastructure without knowing *why* a component is vulnerable and *which* architectural mechanism is compromised.

1.3. The Software-as-a-Graph (SaG) Approach

To bridge the Architecture–Code Gap while resolving the black-box AI challenge, we present **Software-as-a-Graph (SaG)**, an AI-driven pre-deployment **Static System Analysis (SSA)** framework. SaG ingests Architecture-as-Code manifests and executes a four-stage pipeline:

1. **Typed Multigraph Formulation:** SaG models the distributed architecture as a typed, directed multigraph over five core entity types: Applications, Brokers, Topics, Execution Nodes, and Shared Libraries (Section 3.1).
2. **QoS-Aware Logical Dependency Projection:** Using six formal projection rules, SaG derives a semantic `DEPENDS_ON` dependency layer that captures both sequential cascades (via topics and brokers) and simultaneous blast radii (via shared libraries and node colocation), weighted by declared QoS contracts (Section 3.2).
3. **Heterogeneous Graph Learning for Failure Forecasting (Predictive Pathway):** SaG trains a **Heterogeneous Graph Transformer (HGT)** whose relation-specific attention lets a `USES` edge into a shared library propagate differently from a `PUBLISHES_TO` edge into a topic. It forecasts cascading blast radii, ranks critical components, and emits per-relationship criticality along with multi-task reliability/maintainability outputs (Section 4), at 44 ms per 2,000-component system.
4. **Explainable Quality Attribution (Explanation Layer):** To explain *why* a flagged component is critical, SaG combines code-level SCA metrics with topological properties into a deterministic **Reliability–Maintainability (RM)** attribution model (Section 5). Reliability decomposes into **Fault Tolerance** (error propagation depth) and **Availability** (single-point-of-failure exposure), which point to distinct repairs. Being a linear aggregate that models no propagation dynamics by construction, it explains *why* a component is vulnerable rather than how far a cascade travels; its standalone rank correlation is correspondingly modest ($\rho = 0.195$, Section 7.1) — a consequence of that scope, not a defect to tune away.

To ensure methodological rigor, SaG enforces an **input–label independence guarantee**: the learned models and attribution baseline operate strictly on the derived analytical graph G_{analysis}, whereas ground-truth failure impacts are generated by independent discrete-event simulators executing over the raw structural topology $G_{\text{structural}}$ (Section 4.4).

Rationale for Graph Learning vs. Direct Simulation. Because discrete-event simulation $I^*(v)$ defines ground-truth criticality in this study, one might ask: *why train a graph neural network rather than running simulation sweeps directly?* While a complete simulator and an unlimited compute budget would allow simulation to identify critical components in an existing system, four practical reasons make our hybrid graph learning and attribution framework essential:

1. **Handling Unmeasured Components:** Discrete-event simulators only inject faults into active application processes, leaving passive infrastructure (such as message topics or host nodes) without direct simulation labels (30% to 47% of components per system). The learned GNN generalizes across both labeled and unmeasured entities.
2. **Computational Sustainability and CI/CD-Viable Speed:** Cascade simulations are computationally demanding, noisy, and sensitive to seeds and propagation thresholds (label standard deviation across seeds reaches 0.416). The neural network learns a smooth, threshold-marginalized representation while cutting evaluation latency by more than two orders of magnitude — 44 ms on a 2,000-component system against minutes to hours for exhaustive multi-seed simulation. This efficiency is what makes per-commit gating environmentally sustainable and affordable in CI/CD rather than restricted to costly nightly batch sweeps (Section 7.5).
3. **Diagnostic Explainability (Resolving the Black Box):** Simulators and unaugmented GNNs both return only impact — a scalar score or rank. Neither returns root causes in standardized software engineering terms. While a simulator can pinpoint which subscriber lost a feed, it cannot diagnose whether a component is fragile because it is an un-replicated single point of failure or an error cascade hub—diagnoses that demand fundamentally different remediations (host/broker replication versus topic decoupling and circuit breakers). Our ISO-grounded RM model supplies that missing layer: once the predictive path identifies a critical component, the diagnostic path indicates which quality characteristic is at risk and, therefore, which architectural fix applies.
4. **Pre-Deployment Zero-Telemetry Transfer:** Dynamic simulators require runnable containers, operational mock environments, or configured communication harnesses to execute message exchanges.

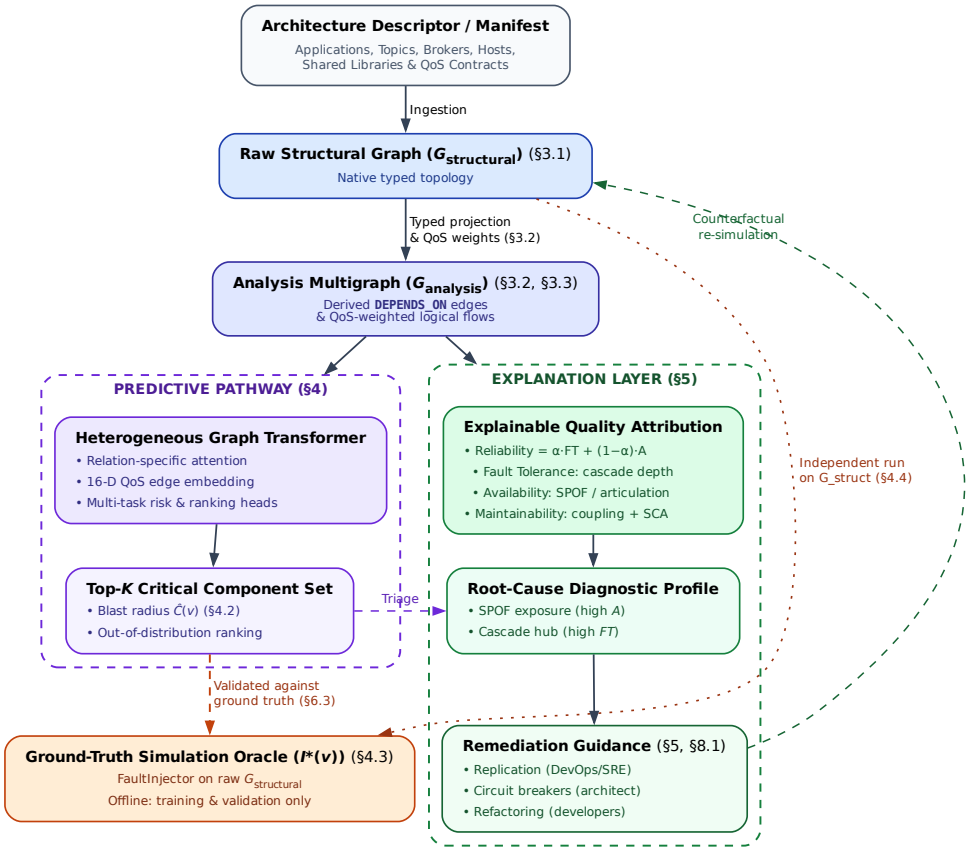

Figure 1: End-to-end architecture of the Software-as-a-Graph (SaG) framework. A shared front end (manifest ingestion → typed multigraph → QoS-weighted `DEPENDS_ON` projection → typed node features) feeds two deliberately separate pathways. The **predictive pathway** (Section 4) is the primary one: it produces a ranked critical set and per-relationship criticality, and is the only pathway validated against the simulation oracle — the oracle scores rankings, which a quality profile is not, and it is strictly an offline training-and-validation component, never a dependency of online inference. The **explanation layer** (Section 5) then produces a standards-grounded quality profile for what the predictor flagged, and the remediation it implies; it explains *why* a component is fragile and is not a ranking model. The single link between them is triage rather than data flow: the architect applies the explanation to whatever the predictor flagged. The two share no parameters, and the oracle runs on $G_{\text{structural}}$ alone, never on the graph the predictors see (Section 4.4). The remediation guidance closes a loop of its own: each candidate edit is re-simulated on its own mutated copy of $G_{\text{structural}}$ and kept only if it beats the simulator's own seed-to-seed noise, before being accepted.

Heterogeneous graph learning enables zero-shot inductive evaluation directly from Architecture-as-Code manifests during continuous integration, assessing topological fragility and performance bottlenecks before provisioning any runtime infrastructure.

1.4. Research Questions

Our empirical evaluation investigates five research questions:

RQ1 (Predictive Efficacy): *How accurately does heterogeneous graph learning predict cascading failure impact and identify the critical component set, compared to traditional, non-learning network metrics?*

RQ2 (Value of Architectural Typing): *Does modeling distinct entity and dependency types (applications, topics, brokers, hosts, and libraries) yield better failure predictions than homogeneous graph models, and does that advantage hold on architectures the model has never seen?*

RQ3 (QoS Encoding, Calibration and Sensitivity): *How do middleware Quality-of-Service policies, the declared weighting constants of the explanation layer, the choice of simulation oracle, and propagation-threshold and normalization settings affect forecasting performance, stability, and explainability?*

RQ4 (Real-World Generalization): *How effectively does the framework transfer zero-shot to real-world, open-source distributed systems across autonomous driving (ROS 2) and cloud-native microservice architectures?*

RQ5 (Analysis Cost and Computational Sustainability): *What does pre-deployment analysis cost at CI/CD time, which pipeline stage dominates that computational footprint, and does the resulting budget support sustainable, per-commit architectural gating?*

1.5. Key Contributions

This paper makes four principal contributions:

1. **Heterogeneous Graph Learning for Pre-Deployment Dependability:** A relation-specific Heterogeneous Graph Transformer that forecasts cascading blast radii from Architecture-as-Code alone, with a 16-dimensional edge feature vector carrying 7 QoS dimensions and multi-task heads for component and relationship criticality (Section 4). Our central claim is about transfer: typing is what lets graph learning generalize to architectures it never trained on, where the untyped alternative collapses (Section 7.2).
2. **A Formal Typed Architecture Model:** A multigraph representation that derives logical dependencies from physical pub-sub linkage and distinguishes sequential cascade propagation from simultaneous multi-consumer library failures, supplying the typed substrate the predictor consumes (Section 3).
3. **A Standards-Grounded Explanation Layer:** An interpretable Reliability–Maintainability model grounded in ISO/IEC 25010/25019 that turns the predictor's ranked output into an actionable diagnosis, separating single-point-of-failure exposure from error-propagation depth — two conditions calling for different repairs (Section 5).
4. **Empirical Benchmark, Real-World Validation, and Sustainability Characterisation:** A rigorous evaluation across seven synthetic topologies (1,545 components) and three real-world systems (Autoware.universe ROS 2, GCP Cloud Microservices, Train-Ticket; 225 components) under strict input–label independence, establishing both where typed graph learning delivers decisive advantages and the boundary where it does not, with a per-stage cost profile proving that the learned AI model is the pipeline's computationally cheapest and greenest stage (Sections 6–7).

Relationship to the authors' prior work. We presented an earlier, shorter version of this work at a peer-reviewed conference [16], focusing on the preliminary typed multigraph formulation and the deterministic quality-attribution model using the synthetic corpus alone. This manuscript is a substantially extended version containing over 70% new technical contributions, fully meeting the Journal of Systems and Software extension policy. Major new contributions include: (1) the complete predictive pathway: the Heterogeneous Graph Transformer, its 16-D continuous-categorical QoS edge encoding, the multi-task masked-loss heads, and all learned results in Section 7; (2) the inductive leave-one-scenario-out (LOSO) protocol and cross-architecture transfer analysis supporting the central typing claim (Section 7.2); (3) zero-shot empirical evaluations across three authentic production systems (Autoware.universe ROS 2, GCP Online Boutique, and Train-Ticket; Section 7.4); (4) an extensive computational cost and sustainability characterisation answering RQ5 (Section 7.5); (5) a formal four-oracle convergent-validity taxonomy and strict input–label independence guarantee preventing data leakage (Sections 4.3–4.4); and (6) global sensitivity analysis across all ten weight constants using Morris screening and Dirichlet simplex sampling (Section 7.3). Material retained from the conference version is limited to preliminary formalisms in Sections 3 and 5, both of which have been thoroughly restructured and expanded. No companion manuscript from this work is under consideration elsewhere.

1.6. Paper Organization

The remainder of this paper is organized as follows: Section 2 reviews related work in distributed systems dependability, performance engineering, static system analysis, and graph representation learning. Section 3 formalizes the Software-as-a-Graph architectural model, the dependency projection rules, and the typed node features both pathways consume. Section 4 presents the Heterogeneous Graph Transformer, its multi-task heads, the simulation oracles that supply its labels, and the input–label independence guarantee. Section 5 presents the interpretable RM explanation layer. Section 6 describes the experimental setup, benchmark corpus, and evaluation protocols. Section 7 presents empirical results for RQ1–RQ5. Section 8 discusses architectural implications, performance and computational sustainability implications, threats to validity, limitations, and concluding remarks.

2. Related Work

This research intersects four foundational domains: distributed systems dependability and performance engineering, static system analysis, software quality models, and explainable graph representation learning.

2.1. Dependability, Performance, and Sustainability in Distributed Systems

The publish–subscribe (pub-sub) and asynchronous event-driven paradigms decouple communicating entities in space, time, and synchronization to achieve elastic scalability and high throughput [5]. Modern middleware standards—such as ROS 2 [1], Apache Kafka [2], DDS [3], and MQTT [4]—govern these exchanges through fine-grained Quality-of-Service (QoS) policies controlling message durability, transport reliability, message priority, and delivery deadlines. In modern cloud-native meshes and distributed AI/LLM serving backbones, asynchronous message passing and queueing topologies form the primary communication substrate, dictating tail latencies, throughput bottlenecks, and resource utilization.

Prior dependability and performance research has focused predominantly on **runtime mechanisms**, including dynamic consensus protocols, broker clustering, adaptive throttling, autoscaling, and automated failover. In parallel, **chaos engineering and runtime verification** [11] inject simulated faults or artificial latency into running staging or production clusters to observe degradation and recovery behaviors. While runtime fault injection provides valuable operational validation, it exhibits critical limitations:

- It requires fully provisioned, operational infrastructure, making it unusable during initial architectural design and pre-deployment planning.
- It introduces operational risks and potential service disruptions if executed on or near live production environments.

- It imposes a substantial **computational and environmental cost**: executing multi-node chaos experiments, distributed traces, and canary deployments consumes vast CPU, memory, and networking resources across cloud clusters, conflicting with modern sustainable computing mandates.

Our work addresses the complementary **pre-deployment phase**: predicting systemic cascading vulnerabilities and performance bottlenecks directly from Architecture-as-Code descriptors before runtime infrastructure is provisioned, enabling zero-runtime, green architectural gating in CI/CD.

2.2. Static Code Analysis (SCA) vs. Static System Analysis (SSA)

Traditional **Static Code Analysis (SCA)** tools (e.g., SonarQube) inspect source code Abstract Syntax Trees (ASTs) within individual services. They evaluate cyclomatic complexity [8], class cohesion, module coupling (e.g., LCOM, CBO) [9, 10], and code duplication to flag internal code smells and defect-prone modules [17, 18, 19, 20]. However, SCA cannot observe runtime communication topology: it is completely blind to inter-service messaging channels, message broker queue saturation, and cross-host failure propagation.

To bridge this "Architecture–Code Gap," **Static System Analysis (SSA)** extends static analysis from single-service source code to the global system architecture. By modeling distributed applications, message topics, brokers, execution nodes, and shared libraries as a connected multigraph, SSA propagates code-level quality metrics across architectural dependencies. This enables engineering teams to identify structural anti-patterns [21, 22] and architectural technical debt [23] early during continuous integration (CI/CD) [24, 25], before defective topologies enter production.

2.3. Software Quality Models and Multi-Criteria Evaluation

Software product quality is standardized by the **ISO/IEC 25010:2023** product quality model [6] and the **ISO/IEC 25019:2023** Quality-in-Use model [7]. ISO/IEC 25010:2023 defines three closely intertwined characteristics critical to modern distributed systems:

- **Reliability:** The degree to which a system performs specified functions under stated conditions, comprising Fault Tolerance and Availability.
- **Maintainability:** The effectiveness and efficiency with which software can be modified, comprising Modularity, Modifiability, and Analysability.
- **Performance Efficiency:** Performance relative to resource consumption under stated conditions, comprising Time Behaviour (latency, response time), Resource Utilization (CPU, memory, bandwidth), and Capacity.

Software engineering measurement explicitly distinguishes between *internal quality* (measured on static artifacts at rest) and *external quality* (measured on executing software systems) [26, 27]. In distributed architectures, architectural debt (such as over-centralized message topics or un-replicated brokers) degrades internal quality while precipitating severe external performance bottlenecks, queue congestion, and catastrophic outages.

Aggregating multi-attribute structural metrics into an auditable quality score is a classic Multi-Criteria Decision Making (MCDM) problem. The **Analytic Hierarchy Process (AHP)** [28] provides a structured pairwise-comparison method with an explicit Consistency Ratio ($CR \leq 0.10$) to ensure weighting schemes remain mathematically sound. In this study, we utilize AHP to construct an audited, explainable Reliability–Maintainability (RM) quality baseline, providing transparent architectural diagnostics alongside our learned graph models.

2.4. Graph Representation Learning and Explainable AI

Network science provides established centrality metrics to identify critical nodes, such as degree, closeness, betweenness centrality [12, 14], articulation points, and PageRank [13, 15]. Foundational studies on network robustness [29], cascading overloads [30], and interdependent networks [31] model how disruptions propagate across connected systems. However, standard network metrics suffer from two major limitations when applied to software architectures:

1. **Dimensional Collapse:** A single centrality scalar cannot distinguish *why* a component is critical—for instance, whether it is a single point of failure (SPOF), an error-propagating cascade hub, or an over-shared library.
2. **Semantic Collapse:** Standard metrics treat all nodes and edges identically. They conflate fundamentally different architectural entities, such as an asynchronous message topic, a shared library, and a physical execution host.

To overcome the limits of hand-engineered metrics, recent research has applied machine learning to network vulnerability (e.g., FINDER [32], DrBC [33], and PowerGraph [34]). However, most existing models rely on **homogeneous message passing** (GCN [35], GraphSAGE [36], GAT [37]), which averages signals across all connections indiscriminately. Because distributed software architectures are inherently **heterogeneous** (comprising distinct entity types and relationship rules), homogeneous models blur critical architectural boundaries and fail to generalize out-of-distribution.

Heterogeneous Graph Neural Networks (RGCN [38], HAN [39], HGT [40], MAGNN [41]) resolve this by employing relation-specific transformations. We build upon the **Heterogeneous Graph Transformer (HGT)** architecture [40] to preserve typed relational semantics when forecasting cascading failure blast radii and performance degradation.

Explainable AI (XAI) vs. The Black-Box Barrier. A critical hurdle identified in applying modern AI to software engineering is the **black-box barrier**: complex neural models output risk scores or continuous embeddings without explaining the underlying structural causality. In production software engineering, an uninterpretable risk ranking is unhelpful: developers cannot determine whether to replicate a host, configure circuit breakers, or refactor shared libraries.

Existing GNN explanation techniques, such as GNNExplainer [42], identify influential subgraphs via edge masking. While useful, these methods explain the model in its own internal latent terms rather than in standardized software engineering concepts. SaG resolves this challenge through a decoupled dual-pathway design: the predictive HGT pathway exposes typed mutual-attention distributions showing *which* architectural relations carried the cascade (Section 7.3), while the deterministic explanation layer attributes fragility to standardized ISO/IEC quality sub-characteristics (Section 5), turning raw predictions into actionable, cost-effective remediations.

Table 1: Comparison of dependability and performance analysis paradigms for distributed systems.

Paradigm	Lifecycle Stage	Topology-Aware	Multi-Typed	Explainable (XAI)	Zero-Runtime Needed	CI/CD Energy Cost
Static Code Analysis (SCA) [8, 9]	Pre-Deployment	No (Single Service)	No	Yes (Code Smells)	Yes	Minimal (Seconds)
Chaos Engineering [11]	Post-Deployment	Yes (Live Cluster)	Partial	Partial (Logs/Traces)	No (Requires Cluster)	High (Cluster-Hours)
Network Centralities [12, 13]	Pre-Deployment	Yes (Flat Graph)	No	No (Single Scalar)	Yes	Low (Seconds)
Homogeneous GNNs [35, 37]	Pre-Deployment	Yes (Flat Graph)	No	No (Black-Box)	Yes	Low (Milliseconds)
Software-as-a-Graph (SaG)	**Pre-Deployment**	**Yes (Multigraph)**	**Yes (5 Types)**	**Yes (ISO/IEC RM)**	**Yes (Manifest-Based)**	**Minimal (44 ms Forward)**

3. The Software-as-a-Graph (SaG) Architectural Model

This section formalizes the Software-as-a-Graph multigraph representation (Section 3.1), the QoS-aware weighting and logical dependency derivation rules (Section 3.2), the dual graph views (Section 3.3), and the typed node feature representations (Section 3.4).

3.1. Formal Multigraph Definition

A complex distributed software system is modeled as a typed, weighted, directed multigraph:

$$\mathcal{G} = (V, E, \tau_V, \tau_E, w_V, w_E) \tag{1}$$

where:

- V is the set of system entities, partitioned into five disjoint entity types:

$$V = V_{\text{app}} \cup V_{\text{broker}} \cup V_{\text{topic}} \cup V_{\text{node}} \cup V_{\text{lib}} \tag{2}$$

- E is the set of directed edges connecting entities.
- $\tau_V : V \to \mathcal{T}_V$ and $\tau_E : E \to \mathcal{T}_E$ are typing functions assigning node and edge categories.
- $w_V : V \to [0, 1]$ and $w_E : E \to [0, 1]$ are weighting functions representing entity criticality and coupling strength.

Table 2: Entity and structural edge types in the SaG model.

Entity Type ($\mathcal{T}_V$)	**Architectural Role**	**Concrete System Examples**
Application (V_{app})	Autonomous process producing/consuming messages	ROS 2 node, Kafka microservice, MQTT client
Broker (V_{broker})	Message routing and queuing intermediary	RabbitMQ exchange, Mosquitto, EMQX broker
Topic (V_{topic})	Named logical communication channel	`/sensor/lidar`, `orders.payment.completed`
Node (V_{node})	Physical host or virtualized environment	Bare-metal server, Kubernetes worker, Cloud VM
Library (V_{lib})	Shared software package or driver dependency	`librdkafka`, OpenCV, Protobuf runtime
Structural Edge ($\mathcal{T}_E$)	**Direction**	**Semantic Meaning**
`PUBLISHES_TO`	App/Library → Topic	Component publishes messages to topic
`SUBSCRIBES_TO`	App/Library → Topic	Component consumes messages from topic
`ROUTES`	Broker → Topic	Broker manages and routes topic traffic
`RUNS_ON`	App/Broker → Node	Process is hosted on physical/virtual host
`CONNECTS_TO`	Node → Node	Physical network link between hosts
`USES`	App → Library	Application links to shared library dependency

Application and Library entities additionally incorporate static code metrics computed via Static Code Analysis (SCA) tools (`cm_*` attributes: lines of code, cyclomatic complexity, coupling between objects, LCOM), thereby linking code-level fragility to topological analysis.

3.2. QoS-Aware Weights and Logical Dependency Derivation

In distributed middleware, communication links vary in coupling strength based on their Quality-of-Service (QoS) contracts. For example, a `RELIABLE` topic with `TRANSIENT_LOCAL` durability binds communicating services more tightly than a `BEST_EFFORT` telemetry stream.

Each topic t carries an intrinsic criticality weight $w(t) \in [0, 1]$ combining its declared QoS semantics with two runtime-stress modulators: payload size and publication frequency:

$$w(t) = \beta \cdot \text{QoS}(t) + \alpha \cdot \text{SizeNorm}(t) + \psi \cdot \text{FreqNorm}(t), \quad (\beta, \alpha, \psi) = (0.75,\ 0.15,\ 0.10) \tag{3}$$

where the QoS term is itself an AHP-weighted aggregate of the declared contract:

$$\text{QoS}(t) = w_{\text{rel}} \cdot q_{\text{rel}} + w_{\text{dur}} \cdot q_{\text{dur}} + w_{\text{prio}} \cdot q_{\text{prio}}, \quad (w_{\text{rel}}, w_{\text{dur}}, w_{\text{prio}}) = (0.24,\ 0.62,\ 0.14) \tag{4}$$

with $q_{\text{rel}}, q_{\text{dur}}, q_{\text{prio}} \in [0, 1]$ denoting the normalized reliability, durability, and transport-priority scores. Durability dominates because it governs whether data survives at all (state persistence across restarts and network partitions), whereas reliability and transport priority both govern in-flight delivery quality, with reliability weighing somewhat more since an unconditional delivery guarantee precedes scheduling. The sub-weight vector is the geometric-mean priority vector of an independently stated Saaty pairwise-comparison matrix, with a small but genuinely non-zero consistency ratio ($CR \approx 0.016$). An earlier version of this matrix was solved backward from a chosen target vector rather than stated independently, so its near-zero CR was an artifact of construction rather than evidence of a real judgment; we report the honest, independently elicited CR here instead. The modulators are logarithmically compressed, $\text{SizeNorm}(t) = \log_2(1 + \text{bytes})/20$ (a 1 MiB design envelope, the practical DDS sample ceiling before RTPS fragmentation dominates) and $\text{FreqNorm}(t) = \log_{10}(1 + \text{Hz})/3$, and $w(t)$ is clamped to $[0.01, 1]$ so that best-effort edges remain visible to graph traversals. Every structural communication edge incident on t (`PUBLISHES_TO`, `SUBSCRIBES_TO`, `ROUTES`) inherits $w_E(e) = w(t)$ together with the topic's QoS vector.

The outer split (β, α, ψ) is a declared convex combination rather than an elicited one, and we report its sensitivity directly (Section 7.3.2) rather than defending the particular triple. A prior KiB-based SizeNorm

divisor of 50 implied an unstated ∼1 EiB envelope and left the term realizing only ∼2% of alpha's declared 15% budget on the evaluation corpus's real payload sizes (32 B–32 KiB); the byte-based 1 MiB envelope above corrects that so alpha's contribution is no longer negligible by construction.

3.2.1. Logical Dependency Projection (`DEPENDS_ON`)

Structural edges capture explicit deployment connections but omit implicit runtime dependencies. For example, a subscriber depends upon a publisher, yet no direct edge connects them in pub-sub architectures. We therefore derive a single unified semantic relation, `DEPENDS_ON`, directed from *dependent* to *dependency* ("if target fails, source is impacted"):

Table 3: The six `DEPENDS_ON` logical dependency projection rules.

Rule	Dependency Category	Structural Pattern (Dependent → Dependency)	Derived Weight (w)
1	`app_to_app`	Subscriber → Publisher (via shared Topic, incl. transitive `USES`)	$1 - \prod_{t \in T}(1 - w(t))$
2	`app_to_broker`	Publisher/Subscriber → Broker routing its topics	$1 - \prod_{t \in T}(1 - w(t))$
3	`node_to_node`	Host → Host (lifted from inter-host app dependencies)	Lifted $\max w$
4	`node_to_broker`	Host → Broker (lifted from hosted app dependencies)	Lifted $\max w$
5	`app_to_lib`	Application → Shared Library it `USES`	$H(w_V(\text{app}), w_V(\text{lib}))$
6	`broker_to_broker`	Broker ↔ Broker (shared-host fate, symmetric)	$w_V(\text{node})$

Rules 1 and 2 aggregate the several topics T mediating one component pair by *probabilistic union* rather than by a maximum, so that additional parallel failure vectors raise coupling monotonically while preserving $w \in (0, 1]$. Rule 5 uses the harmonic mean $H(x, y) = 2xy/(x + y)$ of the consuming Application's and the shared Library's own vertex weights, which calibrates caller criticality against dependency criticality instead of letting either endpoint dominate. Rules 3 and 4 lift the maximum weight of the component-level dependencies crossing the host boundary.

3.2.2. Sequential Cascades vs. Simultaneous Blasts

A key insight of the SaG model is distinguishing between two fundamentally different failure modes:

- **Sequential Cascade (Rule 1):** When an application publisher fails, downstream subscribers experience data starvation. The failure propagates step by step through topics and message queues.
- **Simultaneous Blast (Rule 5):** When a shared software library or execution node crashes, all consuming applications and colocated brokers fail *instantaneously* in a single event.

Retaining entity types and relation-specific dependency rules allows SaG to model both mechanisms, whereas untyped graphs collapse them into identical edges.

On the symmetry of Rule 6. Rule 6 is the one symmetric projection rule, and deliberately so: it does not assert that one broker functionally depends on another, but that two brokers co-located on the same host *share that host's failure domain*. This is the same simultaneous-blast semantics as Rule 5, which is why the derived weight is the shared *Node's* weight rather than either broker's, and why the relation holds in both directions. The mechanism is real in deployed middleware—co-located brokers contend for CPU, page cache, file descriptors, and NIC bandwidth, and a host loss takes both at once, which is precisely why operational guidance for Kafka, RabbitMQ, and EMQX is to spread brokers across fault domains. What Rule 6 does *not* model is intra-cluster broker coupling proper (partition replication, controller or quorum election, federation and shovel links), which is directional and does not require co-location; extending the schema to express it is left to future work. We also note that the rule is close to inert on our corpus: it fires in four of the eight cached scenarios and contributes twelve directed edges in total across 1,770 components, and because the simulation oracles read only $G_{\text{structural}}$ (Section 4.4), it cannot influence any ground-truth label.

3.3. Dual Graph Views and Architectural Layers

The SaG framework maintains two distinct representations of the system:

1. **Structural Graph ($G_{\text{structural}}$):** The raw deployment graph containing physical and structural relations (such as `PUBLISHES_TO`, `ROUTES`, `RUNS_ON`, and `USES`). Discrete-event simulators consume this view exclusively to execute unbiased failure injections (Section 4.3).
2. **Analysis Graph (G_{analysis}):** The projected graph containing derived `DEPENDS_ON` edges annotated with QoS weights and ingested SCA code metrics. All GNN feature representations, graph embeddings, and analytical metrics are computed on G_{analysis}.

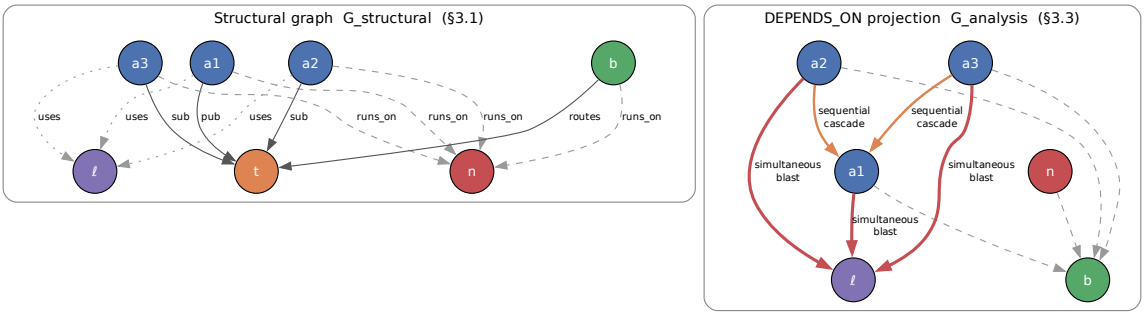

Figure 2: Running example: the raw structural graph (left) and the `DEPENDS_ON` projection derived from it (right). The projection makes implicit runtime dependencies explicit—a subscriber depends on the publishers of its topics even though no structural edge joins them—while the simulators continue to operate on the structural view alone.

G_{analysis} is further organized into four analytical layers (Application, Middleware, Infrastructure, and Global System), allowing architects to evaluate criticality at subsystem levels (e.g., following MIL-STD-498 hierarchical structures).

3.4. Typed Node Feature Representation

Both pathways read the same typed node features off G_{analysis}: the predictor (Section 4) projects them per entity type before message passing, and the explanation layer (Section 5) aggregates them into its quality profile. SaG extracts feature vectors tailored to the 5 entity types:

- **Application ($|V_{\text{app}}|$, 23 dims):** Indices 0–17 represent shared topological metrics (in/out degree, betweenness, closeness, reverse PageRank, clustering coefficient, articulation score, bridge load). Indices 18–22 capture source code metrics extracted via Static Code Analysis (SCA): Lines of Code (LOC), Cyclomatic Complexity, Martin's Instability metric ($I_{\text{code}} = \frac{C_e}{C_a + C_e}$), Lack of Cohesion in Methods (LCOM), and composite Code Quality Penalty (CQP).
- **Library ($|V_{\text{lib}}|$, 25 dims):** Same shared topological (0–17) and code quality (18–22) metrics as Application, plus two library-specific extras (indices 23–24): the size of the transitive reverse-`USES` closure (normalized) and the count of distinct subscribers reachable from that closure's published topics (normalized)—the two structural drivers of a library's blast radius under both simulators' cascade rules that the code-quality metrics alone do not capture.
- **Broker ($|V_{\text{broker}}|$, 19 dims):** Indices 0–17 shared topological metrics; index 18 represents normalized queue buffer capacity.
- **Topic ($|V_{\text{topic}}|$, 22 dims):** Indices 0–17 shared topological metrics; indices 18–21 capture publisher count, subscriber count, log message frequency $\log(1 + \text{freq})$, and ordinal QoS criticality.
- **Infrastructure Node ($|V_{\text{node}}|$, 20 dims):** Indices 0–17 shared topological metrics; indices 18–19 capture normalized CPU core allocation and physical memory (RAM).

4. Graph Learning for Failure-Impact Prediction

Cascading failure impact in distributed software systems is inherently non-linear, multi-hop, and relation-dependent. How far an outage travels depends not merely on how many neighbors a component has, but on *which kind* of architectural relation carries the failure outward and what dependencies lie two or three hops beyond. No closed-form combination of standard centrality metrics can capture these compound dynamics, which is why the primary predictive pathway of Section 1.2 is a learned graph model.

This section presents the Heterogeneous Graph Transformer (HGT) architecture and its typed edge encodings (Section 4.1), the multi-task prediction heads and dimension-masked loss formulation (Section 4.2), the ground-truth simulation oracles (Section 4.3), and the input–label independence guarantee that prevents data leakage (Section 4.4).

4.1. Heterogeneous Graph Transformer Architecture

Because distributed systems comprise heterogeneous entity types (Applications, Libraries, Brokers, Topics, Infrastructure Nodes) and diverse interaction semantics (`PUBLISHES_TO`, `SUBSCRIBES_TO`, `RUNS_ON`, `CONNECTS_TO`, `USES`, `DEPENDS_ON`), we employ a 3-layer **Heterogeneous Graph Transformer (HGT)** architecture [40] with hidden dimension $D = 64$ and $H = 4$ attention heads.

4.1.1. Continuous-Categorical Edge Feature Encoding (16-D)

To capture continuous Quality-of-Service (QoS) constraints and channel semantics, SaG encodes each directed edge $e = (u, v)$ as a 16-dimensional continuous-categorical vector $e_{uv} \in \mathbb{R}^{16}$:

- **Index 0 (Scalar Coupling Weight):** $w_E(e) \in (0, 1]$, defined in Section 3.2 (inherited from $w(t)$ for structural pub/sub edges; or derived via probabilistic union, lifted maximum, or harmonic mean for projected `DEPENDS_ON` edges).
- **Index 1 (Path Count):** Normalized count of simple paths traversing edge e in G_{analysis}.
- **Indices 2–8 (Relation Type One-Hot):** 7-bit one-hot encoding for the structural and derived relations (`PUBLISHES_TO`, `SUBSCRIBES_TO`, `ROUTES`, `RUNS_ON`, `CONNECTS_TO`, `USES`, `DEPENDS_ON`).
- **Indices 9–15 (Explicit QoS Dimensions):** 7 continuous-categorical QoS profile attributes, active for `PUBLISHES_TO` and `SUBSCRIBES_TO` edges (zeroed for other edge types):
  9. *Reliability score* (0.0 = best-effort, 1.0 = reliable).
  10. *Durability score* (0.0 = volatile, 0.5 = transient local, 0.6 = transient, 1.0 = persistent).
  11. *Message priority* ($0.0, 0.33, 0.66, 1.0$ for low, medium, high, urgent).
  12. *Deadline active flag* (0/1).
  13. *Log deadline* $\log_{10}(1 + \text{deadline\_ns}/10^6)$.
  14. *Log max blocking time* $\log_{10}(1 + \text{max\_blocking\_ms})$.
  15. *QoS heterogeneity flag* (1 if the edge's QoS triple deviates from the scenario's modal profile, 0 otherwise).

An edge projection module maps e_{uv} into the hidden space: $e'_{uv} = W_{\text{edge}} e_{uv}$. Prior to relational attention computation, this projection is injected directly into the target node representation: $\tilde{h}_v = h_v + e'_{uv}$.

4.1.2. Type-Specific Projection and Heterogeneous Message Passing

For each source node u and target node v connected by meta-relation $\tau(e) = (\tau(u), \phi(e), \tau(v))$:

1. **Type-Specific Projection:** Node feature vectors x_v (of dimension 19–25 depending on entity type $\tau(v)$) are mapped into the shared D-dimensional hidden space:

$$h_v^{(0)} = \text{LayerNorm}\big(\text{GELU}(W_{\tau(v)} x_v)\big) \tag{5}$$

2. **Relational Mutual Attention:** Type-parameterized Query (Q), Key (K), and Value (V) projections calculate relation-specific attention across H heads:

$$\text{Attn}(u, e, v) = \underset{\forall u \in \mathcal{N}(v)}{\text{Softmax}} \left(\frac{K(u) W_{\text{att},\phi(e)} Q(\tilde{h}_v)^\top}{\sqrt{D/H}} \right) \tag{6}$$

$$\text{Msg}(u, e, v) = V(u) W_{\text{msg},\phi(e)} \tag{7}$$

3. **Bidirectional Message Passing:** To capture downstream consumer starvation and upstream backpressure simultaneously, message passing is executed over both forward and transposed relation views (G_{analysis} and $G_{\text{analysis}}^\top$).
4. **Residual Aggregation and Layer Normalization:**

$$h_v^{(l)} = \text{LayerNorm} \left(h_v^{(l-1)} + \text{Dropout} \left(\sum_{u \in \mathcal{N}(v)} \text{Attn}(u, e, v) \cdot \text{Msg}(u, e, v) \right) \right) \tag{8}$$

4.2. Multi-Task Prediction Heads and Dimension Masking

From the final node embeddings $h_v^{(L)}$, SaG branches into specialized multi-task prediction heads:

- **Reliability Head:** $\hat{R}(v) = \sigma(\text{MLP}_R(h_v)) \in [0, 1]$
- **Maintainability Head:** $\hat{M}(v) = \sigma(\text{MLP}_M(h_v)) \in [0, 1]$
- **Composite Failure Impact Head:** $\hat{I}^*(v) = \sigma(\text{MLP}_C(h_v \parallel \hat{R}(v) \parallel \hat{M}(v))) \in [0, 1]$
- **Relationship Criticality Head:** $\hat{Q}(u, v) = \sigma(\text{TypedEdgeEncoder}_{\phi(e)}(h_u, h_v, e_{uv})) \in [0, 1]$

4.2.1. Dimension-Masked Loss Formulation

The joint optimization objective balances regression accuracy, multi-task dimension learning, ranking fidelity, pairwise ordering, and edge prediction:

$$\mathcal{L} = \mathcal{L}_{\text{composite}} + 0.5 \cdot \mathcal{L}_{\text{dimension}} + 0.3 \cdot \mathcal{L}_{\text{rank}} + 0.1 \cdot \mathcal{L}_{\text{pairwise}} + 0.3 \cdot \mathcal{L}_{\text{edge}} + \lambda_{\text{RM}} \cdot \mathcal{L}_{\text{consistency}} \tag{9}$$

where $I^*(v)$ is the simulated cascade impact defined by the primary oracle (Section 4.3), $\mathcal{L}_{\text{composite}} = \text{MSE}(\hat{I}^*(v), I^*(v))$, $\mathcal{L}_{\text{rank}}$ is the ListMLE ranking loss [43], $\mathcal{L}_{\text{pairwise}}$ is margin-ranking loss, and $\mathcal{L}_{\text{consistency}} = \text{MSE}\big([\hat{R}(v), \hat{M}(v)]_{v \in \text{unlabeled}}, [R_{\text{RM}}(v), M_{\text{RM}}(v)]_{v \in \text{unlabeled}}\big)$ regresses predicted heads toward the diagnostic pathway's baseline (Section 5) on unlabeled nodes. Headline results use $\lambda_{\text{RM}} = 0$, guaranteeing that the predictive and explanatory pathways remain strictly independent.

Dimension Masking: Because dynamic cascade simulation ($I^*(v)$ via `FaultInjector`) observes runtime failure reachability rather than source-code maintainability, maintainability ground truth is unobserved during dynamic simulation. A separate change-propagation oracle $I_M(v)$ evaluates static structural change ripple at the Validate stage, but is never used as a training label to avoid circular supervision. We introduce a boolean dimension mask $m = [m_R, m_M] = [1, 0]$:

$$\mathcal{L}_{\text{dimension}} = \frac{1}{\sum_d m_d} \sum_{d \in \{R, M\}} m_d \cdot \text{MSE}(\hat{d}(v), d^*(v)) \tag{10}$$

This mask ensures the unobserved maintainability head is not artificially penalized or driven toward zero during backpropagation.

4.2.2. Domain-Reweighted Criticality (Q_{domain})

To ground predictions in ISO/IEC 25019 Context of Use ($\vec{\omega} = [q_R, q_M]^\top$), the composite score can be evaluated as:

$$Q_{\text{domain}}(v) = q_R \cdot \hat{R}(v) + q_M \cdot M_{\text{static}}(v) \tag{11}$$

where $M_{\text{static}}(v)$ is drawn directly from the structural analyzer's maintainability baseline, combining learned dynamic reliability with static source-code maintainability. Because maintainability is unobserved in dynamic simulation ($m = [1, 0]$), headline results report $\hat{I}^*(v)$ directly, while domain reweighting sensitivity is evaluated against the static RM baseline in Section 7.3.

4.3. Ground-Truth Simulation Oracles

To evaluate predictive accuracy prior to deployment without relying on production runtime telemetry, SaG executes discrete-event failure simulations over the raw structural multigraph $G_{\text{structural}}$. We establish a formal taxonomy of four component-level oracles and one relationship-level oracle:

- **Cascade Reachability Oracle ($I^*(v)$):** Implemented via `FaultInjector`, this oracle simulates node crashes at component $v \in V$, propagates cascading outages across dependent topics, brokers, and network links via breadth-first dynamic traversal, and calculates the fraction of surviving subscriber feeds severed. Publisher loss is weighted by message publication rate, and resulting feed losses are scaled by a QoS ladder ($\times 1.2$ for `RELIABLE`, $\times 1.15$ for high/urgent priority, $\times 1.05$ for medium) before clamping to $[0, 1]$. $I^*(v) \in [0, 1]$ serves as the **primary continuous target label** for training and evaluating GNN predictors.

- **Multi-Metric Composite Oracle ($I_{\text{comp}}(v)$):** Implemented via `FailureSimulator`, this oracle evaluates a multi-faceted failure impact vector:

$$I_{\text{comp}}(v) = 0.35 \cdot \Delta\text{Reachability} + 0.25 \cdot \Delta\text{Fragmentation} + 0.25 \cdot \Delta\text{Throughput} + 0.15 \cdot \Delta\text{FlowDisruption} \tag{12}$$

  where each term is weighted by operational severity $s(t) = w(t) \cdot \text{rate}(t)$. $I_{\text{comp}}(v)$ serves as the canonical oracle for architectural quality gate verification.

- **Dynamic Queue-Flow Oracle ($I_{\text{dyn}}(v)$):** Implemented via `MessageFlowSimulator` using the SimPy framework [44], this oracle simulates message emission rates, stochastic network latencies, broker buffer saturation, and queue drops under fault injection. It extracts the drop in delivered message rate suffered by surviving consumers.

- **Change-Propagation Oracle ($I_M(v)$):** Implemented via `ChangePropagationSimulator`, this oracle executes a deterministic breadth-first traversal of the reversed dependency graph to quantify maintenance change impact:

$$I_M(v) = 0.45 \cdot \text{ChangeReach}(v) + 0.35 \cdot \text{WeightedChangeImpact}(v) + 0.20 \cdot \text{NormalizedChangeDepth}(v) \tag{13}$$

- **Relationship (Edge) Removal Oracle ($I_{\text{edge}}(u, v)$):** Evaluates the systemic impact of severing an individual dependency or communication channel while keeping endpoint components operational:

$$I_{\text{edge}}(u, v) = I_{\text{comp}}(G \setminus \{(u, v)\}) - I_{\text{comp}}(G) \tag{14}$$

Withholding Declared Topic Criticality from Labels. `FailureSimulator` supports blending a declared `Topic.criticality` label into its severity term, but this is explicitly disabled. Because `Topic.criticality` is an input feature to the GNN (`topic_qos_criticality_ord`), allowing an oracle to consume it would score the predictor against a transformation of its own input features.

Primary Oracle Declaration and Role Assignment. Because the three reliability-facing oracles (I^*, I_{comp}, I_{dyn}) measure distinct operational constructs, we designate $I^*(v)$ **(`FaultInjector`) as the primary**

oracle for all predictive ranking results (Tables 7–9, RQ1–RQ3). $I_{\text{comp}}(v)$ is reserved for Validate-stage quality gates, $I_{\text{dyn}}(v)$ serves as an independent convergent-validity probe, and $I_M(v)$ serves as a structural maintainability reference.

Cross-Oracle Convergent Validity and Critical-Set Bounds. Measured across seven benchmark scenarios and five random seeds on the Application population, mean Spearman rank correlation is $\rho = 0.883$ for (I_{dyn}, I^*), $\rho = 0.468$ for (I_{comp}, I^*), and $\rho = 0.465$ for $(I_{\text{comp}}, I_{\text{dyn}})$. The strong rank agreement ($\rho = 0.883$) between the behavioral queue-flow oracle and the topological cascade injector provides independent convergent evidence across distinct simulation paradigms. However, agreement on the top-K critical set ($K = 0.2n$) is more conservative (mean Jaccard overlap of 0.42 for the strongest pair vs. 0.111 expected by chance), reflecting the intrinsic sensitivity of discrete thresholding in non-linear cascades. Consequently, results established against one oracle are never transferred to another; every evaluation metric explicitly references its underlying simulation oracle.

4.4. Input–Label Independence Guarantee

To eliminate data leakage and ensure rigorous evaluation, SaG enforces strict architectural separation between inputs and labels:

- **Feature Space:** Constructed exclusively from G_{analysis} using static structural topology, static code analysis (SCA) metrics, and declared QoS contracts.
- **Label Space:** Evaluated exclusively on raw $G_{\text{structural}}$ through independent simulation oracles (`FaultInjector`, `FailureSimulator`, `MessageFlowSimulator`).

No simulation outputs, failure trace histories, or dynamic execution telemetry are ever exposed as input features to the GNN or the explanation layer.

5. The Explanation Layer: Standards-Grounded Criticality Attribution

The predictor of Section 4 answers *where* to act. It does not answer *what to do*, and neither does the oracle that scores it — both return impact, not cause attributed in standardised quality terms. A component may be critical because it is a single point of failure, because it propagates errors widely, or because it is a high-coupling maintenance bottleneck, and those diagnoses call for different repairs: replicate the host or broker, decouple the topic, or refactor the module. This section presents the layer supplying that step. SaG decomposes component and relationship criticality into a standards-grounded quality profile, computed over the same typed node features (Section 3.4) but sharing no parameters with the predictor, and applied to whatever the predictor flagged — triage rather than data flow (Figure 1).

5.1. Grounding in ISO/IEC Standards

Following **ISO/IEC 25010:2023** (Product Quality Model) [6] and **ISO/IEC 25019:2023** (Quality-in-Use) [7], we formulate two core formal criticality constructs:

- **Component Criticality (D_1):** The degree to which the sudden failure, unexpected termination, or severe degradation of an individual component reduces the system's capacity to deliver required services within its operational context of use.
- **Relationship Criticality (D_2):** The degree of systemic service degradation resulting from the severance, partitioning, or failure of a specific dependency or communication channel while both endpoint components remain operational.

Criticality is evaluated primarily across two orthogonal quality characteristics: **Reliability (R)** and **Maintainability (M)**. In distributed systems, Reliability and Maintainability degradation directly drives runtime performance collapse and computational energy waste: components with high FT (cascade hubs) generate catastrophic message queue build-ups, head-of-line blocking, and tail-latency spikes under partial

Table 4: The Reliability–Maintainability (RM) quality decomposition.

Dimension	Sub-Characteristic	Architectural Question	Underlying Graph Metrics	Role / Remediation
Reliability (R)	**Fault Tolerance** (FT)	How broadly does failure propagate?	Reverse PageRank on $G^\top$, in-degree, cascade depth	Reliability Eng.: add redundancy
	Availability (A)	Is this a single point of failure?	Directed articulation score (raw + QoS-weighted), bridge ratio, connectivity degradation	DevOps/SRE: replicate host/broker
Maintainability (M)	**Modularity/Modifiability**	How complex and coupled is this?	Betweenness, QoS-weighted out-degree, Code Penalty, coupling-risk imbalance, inverse clustering	Architect: refactor code, decouple

failure, while single points of failure (high A) trigger failover storms, connection thrashing, and redundant retries that consume CPU and cloud resources without completing useful transactions.

Coverage Scope: SaG focuses on Reliability and Maintainability. Safety (which requires domain-specific hazard logs, e.g., ISO 26262 ASIL ratings) and Security (which requires explicit threat models, e.g., STRIDE) are declared external to purely structural analysis and are left for domain-specific extensions.

5.2. Composite Quality Score Formulation

All raw topological and code metrics are rank-normalized to $[0, 1]$. The quality sub-characteristics are formulated hierarchically using the Analytic Hierarchy Process (AHP) [28]:

1. **Fault Tolerance ($FT(v)$):** Measures error cascade potential on the transpose graph $G^\top_{\text{analysis}}$ (where edges follow failure propagation from dependency to dependent):

$$FT(v) = 0.45 \cdot \text{RPR}(v) + 0.30 \cdot \text{Deg}_{\text{in}}(v) + 0.25 \cdot \text{CDPot}_{\text{enh}}(v) \tag{15}$$

   where RPR is Reverse PageRank and $\text{CDPot}_{\text{enh}}$ is an enhanced Cascade Depth Potential term.
2. **Availability ($A(v)$):** Identifies structural single points of failure (SPOFs) across five terms — directed articulation severity, its QoS-weighted variant, edge-level irrecoverability, connectivity degradation, and the component's own QoS weight:

$$A(v) = 0.2563 \cdot \text{AP}_c^{\text{dir}}(v) + 0.1998 \cdot \text{QSPOF}(v) + 0.1998 \cdot \text{BR}(v) + 0.2563 \cdot \text{CDI}(v) + 0.0878 \cdot w(v) \tag{16}$$

3. **Reliability ($R(v)$):** Blends Fault Tolerance and Availability hierarchically:

$$R(v) = \alpha \cdot FT(v) + (1 - \alpha) \cdot A(v), \quad \alpha = 0.36 \tag{17}$$

   Intra-dimension pairwise comparison matrices are audited against Saaty's consistency ratio and measure $CR = 0.001$ (Fault Tolerance), $CR = 0.001$ (Availability), and $CR = 0.000$ (Maintainability) — all well within the $CR \leq 0.10$ acceptability threshold. The shipped intra-dimension weights are a $\lambda = 0.70$ shrinkage blend between the raw AHP-derived vector and a uniform prior (Section 7.3 reports the sensitivity of ranking accuracy to λ).
4. **Maintainability ($M(v)$):** Evaluates structural coupling combined with code-level static analysis across five terms — betweenness, QoS-weighted efferent coupling, the Code Quality Penalty, an afferent/efferent coupling-risk imbalance term, and inverse clustering:

$$M(v) = 0.35 \cdot \text{BT}(v) + 0.30 \cdot w_{\text{out}}(v) + 0.15 \cdot \text{CQP}(v) + 0.12 \cdot \text{CouplingRisk}_{\text{enh}}(v) + 0.08 \cdot (1 - \text{CC}(v)) \tag{18}$$

The baseline composite quality score $Q(v)$ combines both dimensions:

$$Q(v) = 0.80 \cdot R(v) + 0.20 \cdot M(v) \tag{19}$$

When evaluating under a specific ISO/IEC 25019 Context of Use vector $\vec{\omega} = [q_R, q_M]^\top$, the score is reweighted dynamically:

$$Q_{\text{domain}}(v) = q_R \cdot R(v) + q_M \cdot M_{\text{static}}(v) \tag{20}$$

Components are categorized into adaptive criticality tiers using box-plot quartile thresholds:

- **CRITICAL:** $Q > Q_3 + 1.5 \cdot \text{IQR}$
- **HIGH:** $Q_3 < Q \leq Q_3 + 1.5 \cdot \text{IQR}$

- **MEDIUM:** $Q_1 < Q \leq Q_3$
- **MINIMAL:** $Q \leq Q_1$

This enables actionable diagnostics: a service scoring high on A but low on FT is diagnosed as a pure SPOF requiring horizontal replication, whereas a service scoring high on FT is an error cascade hub requiring circuit breakers, queue rate limiting, and bulkhead isolation. Remedying these targeted vulnerabilities not only restores architectural dependability but directly improves performance efficiency and curtails energy-intensive restart storms.

6. Experimental Setup

6.1. Datasets and System Corpus

Our evaluation corpus comprises 1,770 components across ten distributed system architectures:

Table 5: Experimental evaluation corpus.

Dataset / Architecture	System Paradigm	$\|V\|$	$\|V_{\text{app}}\|$	Topics	Brokers	Hosts	Libs	$\|E\|$
Autonomous Vehicle (AV)	ROS 2 Cyber-Physical	152	80	40	4	8	20	797
Enterprise Pub-Sub	Kafka Event Mesh	520	300	120	10	40	50	3,245
Financial Trading	Low-Latency Pub-Sub	124	60	35	5	6	18	580
Healthcare Integration	HL7/FHIR Event Mesh	98	50	25	3	8	12	400
Hub-and-Spoke Enterprise	Broker-Centric Messaging	139	70	30	2	12	25	797
IoT Smart City	MQTT Telemetry Mesh	326	200	80	6	30	10	1,322
Microservices Mesh	Cloud-Native Services	186	90	45	6	15	30	680
Autoware.universe [45]	Real-World ROS 2 Autoware	75	32	24	3	6	10	179
Cloud Microservices [46]	Real-World GCP Boutique	60	22	20	4	6	8	128
Train-Ticket [47]	Real-World Microservices	90	41	30	3	8	8	162
Total		**1,770**						

$|V|$ is the sum of all five entity-type counts per scenario and sums to exactly the corpus total stated above. $|E|$ counts every raw structural relationship instance recorded in the scenario file (`PUBLISHES_TO`, `SUBSCRIBES_TO`, `ROUTES`, `RUNS_ON`, `CONNECTS_TO`, `USES`) — the substrate the simulation oracles traverse — not the denser derived `DEPENDS_ON` projection used for GNN training.

The three real-world architectures are hand-transcribed from open-source repositories using dedicated architectural adapters. The seven synthetic scenarios are produced by a parameterised topology generator: each is fully specified by a committed configuration giving a random seed, per-entity-type counts, seven-number summaries (mean, median, standard deviation, min, max, Q_1, Q_3) for application publish and subscribe fan-out, for applications per host, for library fan-in and for topic payload size, and categorical distributions over the three QoS dimensions. Degree distribution and clustering are *emergent* from those inputs rather than specified directly. Table 6 gives the parameters that determine each topology's shape; the complete configurations are in the replication package.

QoS diversity is not uniform across the corpus. Per-topic QoS is assigned by a domain-keyed lookup (`get_qos_for_topic`) that takes precedence over the categorical distribution declared in each scenario's configuration whenever a domain is set — true of every scenario here. For three domains (`hub-and-spoke`, `microservices`, `enterprise`) that lookup table has a single entry, so every topic receives an identical (reliability, durability, priority) triple regardless of name; a fourth (`av`) collapses four of five entries to the same triple. Table 6's Modal QoS column reflects this directly: those four scenarios show 85–100% topic share at the single modal triple, against 51–79% for the three domains (`finance`, `healthcare`, `iot`) whose lookup tables are genuinely multi-valued. Concretely, $w(t)$ has standard deviation ≈ 0.02 on a $[0, 1]$ scale in the four QoS-flat scenarios, versus 0.19–0.28 in the other three. We report this as a corpus limitation rather than correct it: doing so would change what those four scenarios generate and require regenerating every cached result built on them (Section 8.3).

Table 6: Generative parameters of the seven synthetic evaluation scenarios. Counts, seed and fan-out figures are read directly from the committed configurations. The modal QoS column gives the most common reliability/durability/priority value and the range of topic shares carrying them, computed from the committed topology rather than the config's declared QoS targets, which domain-driven assignment does not always realize (Section 6.1).

Scenario	Config	Seed	Counts	Pub	Sub	Modal QoS (
Autonomous Vehicle (AV)	`scenario_01_autonomous_vehicle`	1001	80/40/4/8/20	2.5	5.0	RELIABLE/T
Enterprise Pub-Sub	`scenario_07_enterprise_xlarge`	7007	300/120/10/40/50	3.0	4.5	RELIABLE/T
Financial Trading	`scenario_03_financial_trading`	3003	60/35/5/6/18	4.0	6.0	RELIABLE/P
Healthcare Integration	`scenario_04_healthcare`	4004	50/25/3/8/12	2.5	3.0	RELIABLE/P
Hub-and-Spoke	`scenario_05_hub_and_spoke`	5005	70/30/2/12/25	2.0	7.0	RELIABLE/T
IoT Smart City	`scenario_02_iot_smart_city`	2002	200/80/6/30/10	2.0	1.5	BEST_EFFO
Microservices Mesh	`scenario_06_microservices`	6006	90/45/6/15/30	1.5	2.0	RELIABLE/T

A note on the ATM case study. One further scenario, an Automated Teller Machine network, serves as an eighth LOSO fold (Section 6.3) and as the subject of the attention analysis (Section 7.3), but is deliberately *not* a row in Table 5 and contributes none of the 1,770 components counted there: it was authored as an illustrative walkthrough, and its configuration lacks the seven-number fan-out summaries the other synthetic scenarios declare. It enters LOSO because that protocol needs held-out topologies rather than characterised ones, so every LOSO figure reports eight folds against a ten-architecture corpus.

6.1.1. Reproducibility of the Corpus

The corpus is regenerable rather than merely archived. Each dataset is produced from its configuration by

```
python cli/generate_graph.py batch -input-dir data/scenarios -output-dir <dir>
```

and a manifest records, per dataset, the seed, the entity counts, the generating commit, and a SHA-256 digest of the emitted topology. A regression test asserts that every committed dataset regenerates *byte-identically* from its configuration and that the manifest matches what is on disk, so a divergence between the published numbers and the published data fails the test suite rather than passing unnoticed. A third party can therefore reproduce the exact graphs these results were computed on, not merely graphs drawn from the same distribution.

6.2. Baselines and Evaluated Predictors

We compare four primary predictor configurations, listing the learned predictors proposed in this paper first and the training-free baselines they are measured against second:

1. **HGL / HGL-QoS** (proposed): Relation-specific Heterogeneous Graph Transformers (Section 4), without and with the explicit 7-dimensional QoS edge encoding.
2. **GL / GL-QoS** (proposed ablation): Homogeneous Graph Attention Networks (GAT) [37] trained on the flattened, untyped graph projection — the controlled comparison that isolates what relation-specific typing buys.
3. **Topo-QoS** (baseline): QoS-weighted topological centrality baseline, training-free.
4. **Topo-BL** (baseline): Unweighted structural centrality (betweenness centrality and articulation point scoring), training-free.

The out-of-distribution table (Table 9) additionally reports **RM** / $Q(v)$ (the deterministic hierarchical quality attribution model of Section 5) as a non-competing diagnostic reference point, not as a fifth predictor: RM is not fitted to rank components, and its row exists to show how much the predictive path adds over static attribution (Section 1.2), not to compete on ranking accuracy. The same deterministic RM scorer is also the instrument behind every sensitivity sweep in Section 7.3: because RM is a closed-form function of its declared constants, sweeping those constants isolates their effect from training variance in a way the learned predictors cannot.

6.2.1. Evaluation Substrate

Predictors in this comparison are not all evaluated over the same graph view, and the distinction matters for reading the tables that follow. **Topo-BL, Topo-QoS, GL and GL-QoS** are scored on the derived Application–Library `DEPENDS_ON` projection (Section 3.2) — the same substrate built for GNN feature/label alignment and discussed further in Section 7.3 — while **HGL and HGL-QoS** consume the full native typed multigraph over all five entity types. **No predictor in either group reads $G_{\mathbf{structural}}$**: that graph remains the exclusive substrate of the ground-truth simulation oracles (Section 4.4), and the independence guarantee enforced by `tests/test_independence_guarantee.py` holds identically for every variant in this section.

The projection substrate was adopted for the homogeneous and untrained baselines because the full native pub-sub graph is not learnable by an untyped model on this corpus: Application nodes carry near-zero betweenness and bridge-ratio in the raw graph (they never route messages), producing degenerate, near-constant feature vectors, and a single homogeneous message-passing layer over-smooths every Application into an identical representation by aggregating the same high-fan-in Topic hub. This is the design rationale recorded in the replication package rather than a measured ablation — we did not run GL/GL-QoS on the native substrate to quantify the collapse directly, and do not report a number for it.

This asymmetry is a scope condition on the RQ2 comparisons in Section 7.2: part of HGL's advantage over GL reflects the wider native node set the typed model can consume, not relation-specific typing alone, since the untyped baseline could not be evaluated on that same view. It does not, however, affect which nodes are scored: the evaluation population is pinned identically across every variant, resolved from the native graph and the simulation labels alone — never from a variant's own substrate or predictions — so no comparison in this paper is confounded by which nodes a particular predictor happened to see (Section 6.3).

6.3. Evaluation Metrics and Protocols

- **Ranking Accuracy:** Spearman rank correlation (ρ) and Kendall's tau (τ) between predicted rankings and ground-truth simulated impact $I^*(v)$, the primary oracle declared in Section 4.3.
- **Critical-Set Identification:** $F_1@K$, Precision@K, and Recall@K for top-K critical components, where K is 20% of the evaluated node population, rounded to nearest. Because the predicted and true sets both contain exactly K elements, precision, recall and F_1 coincide at K; the figure is a top-K overlap and we read it as such.
- **Statistical Significance:** Paired Wilcoxon signed-rank tests [48] ($p < 0.05$) and bootstrap 95% confidence intervals ($B = 2,000$) [49, 50].

6.3.1. Evaluation Population

Every predictor in a given table is scored on an identical node set, resolved from the graph and the labels alone — never from any variant's predictions — so that no comparison is confounded by which nodes a particular method happened to score. Unless stated otherwise that set is the **Application** population: it is the population the framework's central claim is about (topology predicts application-layer cascade criticality), and it is the only one every variant can score. This matters more than it may appear. Pooling node types into a single correlation mixes populations with different impact scales and base rates, and on this corpus that pooling is not benign — it moves the RM composite's rank correlation *outside* the range spanned by its own per-type correlations (Section 7.3). We therefore report stratified, single-population figures throughout, and flag explicitly wherever a pooled figure appears.

6.3.2. Evaluation Protocols

- **In-Distribution Evaluation:** 60% train / 20% validation / 20% test node splits pinned by node identity within each scenario, evaluated over five random seeds $\{42, 123, 456, 789, 2024\}$.
- **Inductive Leave-One-Scenario-Out (LOSO):** Across all eight cached scenarios (the seven synthetic scenarios plus the ATM case study, Section 7.3), models are trained on the remaining seven and evaluated zero-shot on the held-out scenario, for eight folds total, to test out-of-distribution generalizability.

- **Real-World Architectural Transfer:** Evaluating zero-shot transfer on authentic open-source architectures.

7. Results and Empirical Analysis

7.1. RQ1: Graph Learning vs. Structural Baselines

Table 7 presents the in-distribution held-out Spearman rank correlation (ρ) against simulated cascade impact $I^*(v)$ across all seven synthetic scenarios.

Table 7: In-distribution held-out Spearman rank correlation (ρ) against $I^*(v)$ (seed means over 5 seeds with bootstrap 95% CIs; n is held-out Application count).

Scenario	n	Topo-BL	Topo-QoS	GL	GL-QoS	HGL	HGL-QoS
AV System	16	0.283	0.701	0.764	0.678	**0.789**	0.649
Enterprise	60	0.420	0.789	0.871	0.483	0.880	**0.891**
Financial Trading	12	0.289	0.700	0.645	0.539	**0.854**	0.770
Healthcare	10	−0.101	**0.772**	0.623	0.463	0.652	0.645
Hub-and-Spoke	14	0.234	0.359	**0.547**	0.054	0.534	0.297
IoT Smart City	40	−0.063	0.073	0.289	0.291	**0.881**	0.842
Microservices	18	0.401	**0.707**	0.525	0.564	0.483	0.476
Mean	—	**0.209**	**0.586**	**0.609**	**0.439**	**0.725**	**0.653**

Table 8: Paired Wilcoxon signed-rank tests across scenarios ($n = 7$, two-sided).

Comparison	$\Delta\rho$	Won	Wilcoxon W	p-value	Significance
HGL vs. Topo-BL	**+0.516**	7/7	0.0	**0.0156**	**Statistically Significant** ($p < 0.05$)
HGL vs. GL-QoS	**+0.286**	6/7	1.0	**0.0312**	**Statistically Significant** ($p < 0.05$)
HGL vs. Topo-QoS	**+0.139**	5/7	9.0	0.469	Not significant
GL vs. Topo-QoS	**+0.023**	4/7	10.0	0.578	Not significant
HGL vs. GL	**+0.116**	5/7	7.0	0.297	Not significant
HGL-QoS vs. HGL	**−0.072**	1/7	3.0	0.078	Not significant (marginal; HGL-QoS trails in-distribution)

7.1.1. Out-of-Distribution (LOSO) Generalization

Under inductive Leave-One-Scenario-Out (LOSO) cross-validation, the model must predict cascading criticality on an unseen system topology:
Eight LOSO folds are reported (the seven synthetic scenarios plus the ATM case study of Section 7.3), each holding out one scenario and training on the remaining seven. Every variant is scored on the identical Application node set per fold (Section 6.3); paired Wilcoxon tests below are over the eight folds.

Key Insights for RQ1:

1. **The typed predictor is the best configuration overall, and its margin over every learned alternative is decisive.** HGL-QoS leads on both metrics ($\rho = 0.608$, $F_1@K = 0.414$), with a large, significant margin over both homogeneous GNNs (+0.246 over GL-QoS, 8/8 folds, $p = 0.0078$).
2. **Critical-set detection is where the learned model separates from the untrained baselines.** HGL-QoS improves $F_1@K$ over Topo-QoS by +0.034 (0.414 vs. 0.380, a 8.9% relative gain) and over GL by +0.177. The advantage is real but far smaller than a pooled-population reading of the same experiment suggests.
3. **QoS encoding is what carries the typed model out of distribution.** HGL-QoS leads HGL by $\Delta\rho = +0.169$, winning 7 of 8 folds ($p = 0.0156$) — a larger and more consistent effect than the in-distribution comparison in Table 8 suggests in the opposite direction. See Section 7.3.

Table 9: Inductive Leave-One-Scenario-Out (LOSO) evaluation, Application population, eight folds. Rows are grouped by role, not listed as competing variants: training-free structural baselines, the learned predictors proposed in this paper, and the RM/$Q(v)$ diagnostic reference of Section 1.2, which is not a ranking model (Section 5.1) and is included only to quantify how much the predictive path adds over static attribution.

Predictor / Reference	Mean LOSO ρ	Std ρ	Critical-Set $F_1@K$	Requires Training
Training-free structural baselines				
Topo-BL	0.301	0.126	0.363	No
Topo-QoS	0.571	0.181	0.380	No
Learned predictors				
GL (Homogeneous)	0.086	0.122	0.237	Yes
GL-QoS (Homogeneous)	0.363	**0.089**	0.341	Yes
HGL (Typed Heterogeneous)	0.439	0.145	0.327	Yes
HGL-QoS (Typed + QoS)	**0.608**	0.143	**0.414**	Yes
Diagnostic reference — not a ranking model				
RM / $Q(v)$	0.195	0.130	0.327	No

4. **The boundary: on ranking alone, the untrained QoS baseline is not beaten.** Against *Topo-QoS*, HGL-QoS's margin is +0.037, won in only 5 of 8 folds, and **not statistically significant** ($p = 0.64$). We state this plainly: on out-of-distribution Application ranking we cannot claim heterogeneous graph learning outperforms a well-constructed structural baseline, only that it matches it. The case for the predictive pathway rests on the three findings above and on what it supplies that a centrality score cannot — typed attention, per-relationship criticality, multi-task quality outputs — not on ranking accuracy alone.
5. **The explanation layer is weakly predictive, not anti-predictive.** RM/$Q(v)$ reaches $\rho = 0.195$ under distribution shift — better than untyped GL, worse than every QoS-aware variant. Its value is interpretable attribution rather than ranking (Section 5), but it is not noise.

A note on population. An earlier version of this table pooled every node type carrying a simulated label, which inverted several conclusions: it placed the RM baseline at $\rho = -0.014$ rather than +0.195 and GL at 0.381 rather than 0.086. This is the Simpson's-paradox hazard analysed in Section 8.3, and it is why every figure here is reported on a single, stated population.

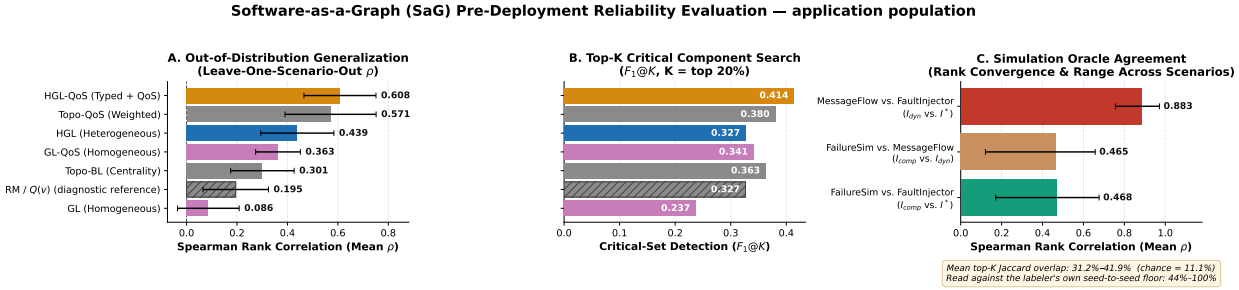

Figure 3: Results at a glance, all on the Application population. **(A)** Out-of-distribution rank correlation per variant, whiskers showing σ across the eight LOSO folds; note that the training-free Topo-QoS baseline places second. The hatched bar is RM/$Q(v)$, the diagnostic reference of Section 1.2 — shown for context, not as a competing ranking predictor. **(B)** Critical-set detection at $K = 20\%$, same hatching convention. **(C)** Agreement between the three simulation oracles, whiskers showing the observed range across scenarios; the annotation gives top-K set agreement against both chance and the labeler's own seed-to-seed floor.

7.2. RQ2: Value of Typed Heterogeneity

To evaluate the contribution of node and edge typing, we compare relation-specific HGL against homogeneous GL across different validation regimes:

- **In-Distribution:** Heterogeneous HGL leads homogeneous GL by $\Delta\rho = +0.116$ (0.725 vs. 0.609; not statistically significant, $p = 0.297$, 5/7 scenarios won). On familiar topologies, homogeneous GNNs can partially approximate type distinctions through structural degree signatures, so the typed model holds only a point-estimate edge. The contrast with the out-of-distribution result below is the finding: typing buys little when the topology is already known and a great deal when it is not.

- **Out-of-Distribution (LOSO):** This is where typing decides the outcome. Heterogeneous HGL outperforms homogeneous GL by +0.353 ($\rho = 0.439$ vs. 0.086), winning *all eight* folds (paired Wilcoxon, $p = 0.0078$), and the gap between the QoS-aware variants is +0.246 (HGL-QoS 0.608 vs. GL-QoS 0.363), also 8/8 and $p = 0.0078$. The untyped model very nearly fails to transfer at all on Application ranking: at $\rho = 0.086$ it is below even the unweighted structural baseline (0.301). When encountering unseen topologies, relation-specific message passing is not an incremental refinement but the difference between a model that generalizes and one that does not.

A scope condition on this comparison. GL and HGL are not evaluated on the same graph view: as declared in Section 6.2.1, GL/GL-QoS run on the Application–Library `DEPENDS_ON` projection because the homogeneous model collapses to constant output on the full native graph, while HGL/HGL-QoS consume that native graph directly. Part of the gap above therefore reflects the wider node population and relation set available to the typed model, not relation-specific message passing in isolation. We did not run a native-substrate GL ablation to separate these two effects — doing so would require resolving the collapse the projection substrate exists to avoid — so the +0.353 figure should be read as the advantage of the typed pathway as deployed, not as an isolated effect of typing alone.

7.2.1. Empirical Edge-Removal Analysis

We further probed edge criticality by simulating the removal of candidate relationship channels while keeping both endpoint components alive. On the `av_system` topology across 50 candidate bridge edges, 46 edges exhibited zero downstream cascade impact, while the 4 edges with non-zero impact were direct library communication channels (`PUBLISHES_TO` / `SUBSCRIBES_TO`). This demonstrates that individual communication links are largely redundant, whereas component-level failures and shared library dependencies induce disproportionate systemic disruption.

7.3. RQ3: Ablations and Sensitivity Analysis

A note on what ρ measures here. Several sweeps below — AHP shrinkage λ, the topic-weight triple, the QoS sub-weights, the Morris and Dirichlet joint sweeps — report RM's rank correlation against $I^*(v)$ as their outcome. RM is not proposed as a ranking model (Section 5.1), so within this subsection ρ is used strictly as a *sensitivity probe*: does perturbing a declared constant move a measurable quantity, and by how much relative to the gaps between predictor families elsewhere. A sweep finding one setting higher than another is evidence about that constant's leverage, not an optimality claim about RM as a ranker.

7.3.1. QoS Feature Encoding

Explicitly encoding continuous QoS attributes in the GNN (HGL-QoS) does not yield a uniform effect — it trades a little in-distribution accuracy for a large out-of-distribution gain. The two directions are summarised below; note that they are measured under different protocols on different fold counts, though both are scored on the Application population.

Table 10: HGL-QoS against HGL under both protocols. Both are Application-scored; the regimes differ in fold count (7 in-distribution scenarios vs. 8 LOSO folds, the latter including the ATM case study).

Protocol	**HGL**	**HGL-QoS**	$\Delta\rho$	**Folds won**	**Wilcoxon p**
In-distribution (Table 7)	0.725	0.653	−0.072	1/7	0.078 (n.s.)
Inductive LOSO (Table 9)	0.439	**0.608**	**+0.169**	**7/8**	**0.016**

The apparent contradiction between the two tables is a genuine regime effect, not an inconsistency. In-distribution, HGL-QoS *trails* the base typed model, but the deficit is not significant and is small relative to its own scatter (across-scenario $\sigma = 0.35$ against a 0.072 gap), concentrated in two structurally atypical topologies, Hub-and-Spoke (-0.238) and AV (-0.140), with the remaining five within ± 0.084. This is consistent with QoS features adding redundant signal, and mild overfitting risk, on topologies already seen — the derived `DEPENDS_ON` graph already embeds much of the same routing and coupling information in its edge weights.

Under LOSO the relationship reverses decisively and *is* significant: $+0.169$, winning 7 of 8 folds. When the topology is unseen, structural degree signatures no longer transfer, whereas a declared QoS contract means the same thing in an unfamiliar architecture as a familiar one — `RELIABLE` and `PERSISTENT` carry their semantics across deployments in a way a betweenness percentile does not. QoS encoding is therefore *situational* rather than redundant: insurance against distribution shift, bought at a small, insignificant in-distribution cost. The practitioner rule: HGL when the target resembles the training corpus, HGL-QoS when it does not, and HGL-QoS by default when that is unknown.

7.3.2. Topic-Weight Coefficients (β, α, ψ)

The outer split of Equation 3 is a declared convex combination, not an elicited one, so we measure what rests on it rather than argue for the particular triple. We swept (β, α, ψ) over a grid spanning the whole simplex — including the QoS-only corner $(1, 0, 0)$ and a uniform prior $(\frac{1}{3}, \frac{1}{3}, \frac{1}{3})$ — propagating each point through the derived `DEPENDS_ON` edge weights into two closed-form scorers, against the revised SizeNorm envelope and re-elicited QoS sub-weights (Section 3.2).

Table 11: Sensitivity of the topic-weight ordering and of downstream rank correlation against $I^*(v)$ to the declared coefficients (β, α, ψ). Application population, mean over seven scenarios; 375 topics.

(β, α, ψ)	ρ **of** $w(t)$ **vs. shipped**	**Topo-QoS** ρ	**RM** ρ
$(0.75, 0.15, 0.10)$ *shipped*	1.000	0.604	0.267
$(0.90, 0.05, 0.05)$	0.966	0.600	0.273
$(0.85, 0.15, 0.00)$	0.950	0.608	0.268
$(1.00, 0.00, 0.00)$	0.852	0.618	0.268
$(0.60, 0.20, 0.20)$	0.970	0.604	0.261
$(0.50, 0.25, 0.25)$	0.942	0.603	0.259
$(\frac{1}{3}, \frac{1}{3}, \frac{1}{3})$ *uniform*	0.870	0.608	0.256
Spread across grid	—	**0.018**	**0.017**

The coefficients are not load-bearing. Across the entire simplex the induced ordering of $w(t)$ never falls below $\rho = 0.852$ against the shipped ordering, and downstream rank correlation moves by at most 0.018 (Topo-QoS) and 0.017 (RM) — an order of magnitude below the gaps between predictor families. Over the 375 corpus topics the QoS term contributes a weighted standard deviation of 0.194, the frequency term 0.021, the payload term 0.020; this corrects an earlier KiB-based SizeNorm divisor of 50 (an unstated $\sim$1 EiB envelope) under which the payload term contributed only 0.0039, realizing $\sim$2% of alpha's declared budget. The claim is that the declared triple drives no reported result, *not* that it is optimal: the QoS-only corner is marginally better for both Topo-QoS (0.618 vs. 0.604) and RM (0.268 vs. 0.267). The inner QoS split $(0.24, 0.62, 0.14)$ is the geometric-mean priority vector of an independently stated Saaty matrix (CR ≈ 0.016), replacing an earlier matrix solved backward from a target vector; both are pinned by regression tests.

7.3.3. Intra-Dimension AHP Weight Shrinkage

We evaluated the sensitivity of the RM attribution baseline by sweeping a shrinkage parameter $\lambda \in [0, 1]$ blending internal term weights from uniform prior ($\lambda = 0$) to calibrated AHP judgement ($\lambda = 1$). Note that λ shrinks only the *intra-dimension* vectors: the composite weights $(q_R, q_M) = (0.80, 0.20)$ are declared constants and are identical at every λ.

Table 12: Sensitivity of RM rank correlation against $I^*(v)$ to the AHP shrinkage parameter λ, Application population, mean over seven scenarios.

λ **Setting**	**0.00 (Uniform)**	**0.50**	**0.70 (Default)**	**0.80**	**1.00 (Raw AHP)**
Mean Rank Correlation (ρ)	**0.348**	0.291	0.267	0.256	0.232

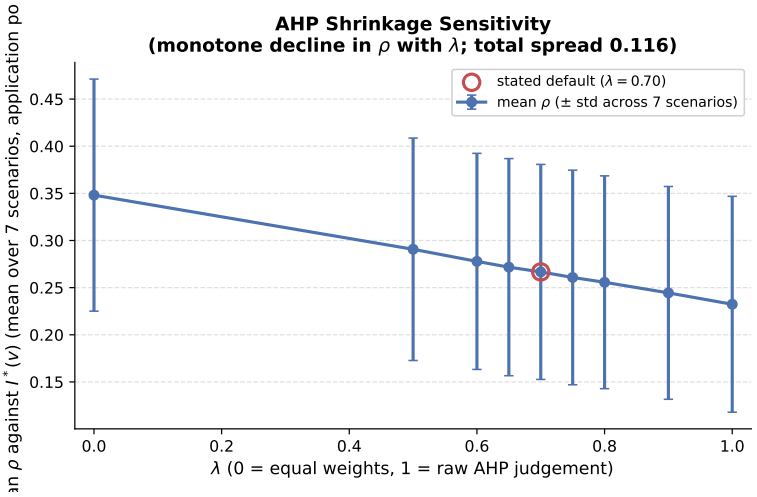

Figure 4: Sensitivity of the RM composite's rank correlation against $I^*(v)$ to the AHP shrinkage parameter λ, Application population, mean over seven scenarios. ρ declines monotonically toward the elicited judgement: the uniform prior at $\lambda = 0$ is the best setting in the sweep.

λ is the most consequential of the ten constants on this probe, and moving it toward the elicited judgement does not raise ρ. ρ declines monotonically from the uniform prior toward elicited judgement (Figure 4): the uniform prior scores highest (0.348 against 0.232 at raw AHP), winning all seven scenarios (paired Wilcoxon, $p = 0.0156$), and the shipped $\lambda = 0.70$ trails it by $\Delta\rho = -0.081$ ($p = 0.0156$), with no plateau around the default. Read as the sensitivity probe this subsection declares — not a ranking-optimality claim about RM (Section 5.1) — the elicited hierarchy buys *transparency*, making the composite auditable and its terms nameable, rather than ranking accuracy. We retain $\lambda = 0.70$ on that basis.

A measurement correction. An earlier version scored $Q(v)$ from features restricted to the Application–Library `DEPENDS_ON` projection rather than the full typed graph. There no Application is an articulation point and no incident edge a bridge in six of eight scenarios, so four of Availability's five terms vanish and $A(v)$ — carrying $w_R(1-\alpha) \approx 0.51$ of the composite — is constant, and the sweep returned uniformly negative ρ (-0.146 to -0.111). Figures above use the full pipeline (Section 5.2); both series are retained in the replication artifact. The same correction applies to the threshold and domain-weighting ablations below.

7.3.4. Joint Sensitivity Across All Ten Weight Constants

The sweeps above, like every other ablation here, vary one declared constant at a time. That one-at-a-time (OAT) design cannot detect interactions between factors, or say whether a factor flat in isolation stays flat when a second moves with it [51]. We closed that gap by sweeping all ten hand-set weight constants jointly — (β, α, ψ), $(w_{\text{rel}}, w_{\text{dur}}, w_{\text{prio}})$, the vertex power-mean exponent p, the library fan-out coefficient γ, the AHP shrinkage λ, and r_α — under two complementary designs on six scenarios (`enterprise_system` excluded on cost grounds, its structural analysis costing roughly 15× any other per evaluation point).

Morris elementary-effects screening [52, 53] takes 10 random trajectories through the full ten-dimensional hyper-rectangle (110 evaluations), deliberately *not* constraining the convex-combination groups to a unit sum, and ranks factors by μ^*, the mean absolute elementary effect on mean ρ against $I^*(v)$: λ and r_α dominate, consistent with the OAT sweeps above. The topic-weight and QoS sub-weight triples sit

Table 13: Morris elementary-effects screening over all ten weight constants, ranked by influence (μ^*) on mean ρ. Six scenarios, 10 trajectories, 110 evaluations.

Factor	μ^*	σ
λ (AHP shrinkage)	0.124	0.077
r_α	0.096	0.035
α	0.019	0.034
w_{rel}	0.019	0.018
β	0.017	0.023
w_{dur}	0.014	0.017
w_{prio}	0.013	0.018
ψ	0.012	0.015
p (power-mean exponent)	0.005	0.005
γ (fan-out)	0.001	0.002

in a comparable, modest band ($\mu^* \in [0.012, 0.019]$) rather than one dominating the other by an order of magnitude — confirming jointly what the corrected term contributions show at the formula level: none of these six factors is individually load-bearing, and none negligible by construction. p and γ are least influential. σ relative to μ^* is largest for λ and α, indicating their effect depends on where the other nine sit — the interaction an OAT sweep cannot diagnose, though within the same order as μ^* rather than dominating it.

Dirichlet simplex sampling asks the sharper question: over the realistic space of declared budgets — respecting the unit-sum constraint on both groups, other factors shipped — how much do accuracy and the shortlist vary. Over 100 draws, mean $\rho = 0.243$ (shipped 0.252), sd = 0.006, range $[0.229, 0.260]$, 90% interval $[0.231, 0.253]$; rank stability is high (mean Kendall $\tau = 0.899$, worst 0.791; mean top-20% Jaccard 0.825, worst 0.699). The declared budgets are, jointly and on the realistic simplex, not a lever on either the score or the shortlist.

7.3.5. Convergent Validity Across Simulation Oracles

To test construct validity, we compared component criticality rankings across three oracles of Section 4.3: $I^*(v)$ (`FaultInjector`), $I_{\text{comp}}(v)$ (`FailureSimulator`), and $I_{\text{dyn}}(v)$ (`MessageFlowSimulator`), evaluated across seven scenarios and five seeds on the Application population. The Maintainability-only oracle $I_M(v)$ is excluded as it targets a different dimension; its own limitation is discussed in Section 8.3.

Table 14: Inter-oracle agreement. Chance-level top-K Jaccard at $K = 0.2n$ is 0.111; the tie-robust column admits every component tied with the K-th.

Oracle pair	**Mean ρ (range)**	**Mean τ**	**Jaccard@K**	**Tie-robust**
I_{dyn} vs. I^*	**0.883** (0.756–0.972)	0.745	0.424	0.419
I_{comp} vs. I^*	0.468 (0.171–0.677)	0.349	0.307	0.312
I_{comp} vs. I_{dyn}	0.465 (0.121–0.658)	0.348	0.313	0.313

Ordering converges; the critical set does not. The behavioural queue simulator and the topological cascade oracle agree strongly on ordering ($\rho = 0.883$, never below 0.756). Because the two are built on different principles — delivered message rates under load versus reachability over edges — this is not reducible to a shared construction artifact, and is the strongest convergent evidence that $I^*(v)$ tracks dynamic service disruption rather than only topology. The composite oracle is the outlier ($\rho \approx 0.47$).

Set-level agreement is weaker, and we report it as the binding limitation. Top-K Jaccard is 0.42 for the strongest pair and 0.31 for the others, against 0.111 under independent rankings. Two controls precede any interpretation. It is *not* a tie-breaking artifact: admitting every component tied with the K-th moves it by at most 0.005. Nor is it simply label noise, though noise is a substantial part — the labeler's own seed-to-seed self-agreement spans 0.44–1.00, so one oracle against *itself* reaches only 0.44 in its worst scenario. Top-K identity at $K = 0.2n$ is intrinsically unstable, and the cross-oracle values sit just below the floor a single

oracle achieves against its own reruns. The defensible claim is about ranking, not membership of a fixed-size set.

A falsified hypothesis, reported. We tested whether weighting both topological oracles by the same $w(t)$ makes them converge — the mechanism by which QoS weighting would be a construct-validity improvement rather than a modelling choice. It does not: $\rho(I_{\text{comp}}, I^*)$ is 0.468 weighted and 0.477 unweighted ($\Delta\rho = -0.009$; Jaccard 0.307 vs. 0.306). We record the negative result rather than leaving the artifact uncited.

Two reasons that null is weaker than it looks. First, the arms are not comparably strong: disabling QoS removes I_{comp}'s severity term entirely, but on I^* removes only a ladder multiplier that a $[0, 1]$ clamp already suppresses near feed-loss saturation — exactly the high-impact regime a ranking metric is most sensitive to. Toggling that ladder shifts I^*'s labels by mean absolute 0.064 (range 0.012–0.145) against 0.013 for a shared $w(t)$-proportional treatment, so the null bounds *this* treatment rather than QoS-aware labelling in general; the untested arm is I_{dyn}'s delivery-rate label. Second, four of the seven scenarios have $w(t)$ standard deviation ≈ 0.02 (Section 6.1), so a null measured partly on a QoS-flat substrate is expected regardless of treatment strength. We did not separate the two effects, and flag this as an open confound (Section 8.3).

7.3.6. Domain-Specific Weighting Sensitivity

We swept the composite reliability weight w_R over its full range across 10 scenarios, reading off the shipped static default ($w_R = 0.80$), an equal split (0.50), and the domain-derived value. The three are indistinguishable (mean $\rho = 0.353$, 0.349, 0.347), and the sweep explains why: over the entire range mean ρ moves only from 0.341 to 0.365, a total spread of 0.024. The derived w_R lies in $[0.70, 0.76]$ across domains, and mean Kendall fidelity between domain-derived and static rankings is $\tau = 0.974$. This confirms the scoping of Section 5.1: Context-of-Use reweighting is an *attributional* mechanism, not a ranking-improvement device, and should be reported as neither gain nor loss.

7.3.7. Threshold and Normalization Sensitivity

We swept 7 scenarios across a cascade propagation-threshold parameter $\in \{0.0, 0.1, 0.2, 0.35, 0.5, 0.75, 1.0\}$ controlling the oracle's eligibility cutoff, and separately across 3 label-normalization techniques (robust, min-max, standard) at fixed threshold. Ranking is more sensitive to the threshold ($\Delta\rho = 0.102$; mean ρ rises from 0.177 at 0 to 0.278 at 0.5 then plateaus, the shipped 0.35 sitting inside that plateau at 0.274) than to normalization ($\Delta\rho = 0.022$; robust 0.267 against 0.244 for both alternatives). The threshold is the more consequential free parameter, and the shipped setting sits on the flat part of its curve rather than at a tuned peak. A small spread means the ordering is stable under that parameter — not, by itself, that it is *correct*.

7.3.8. Availability Was Measuring Almost Nothing, and Why

The Availability sub-characteristic $A(v)$ (Section 5) was, in the version these results were first computed on, driven almost entirely by one component of a five-term formula that fires vanishingly rarely here. AP_c(directed), Bridge Ratio and CDI are all articulation-point-gated: a node that is not a Tarjan cut vertex scores exactly 0.0 on all three by construction. On six of eight LOSO scenarios — including the 350-component Enterprise topology — *zero* Application-layer components are articulation points, so all three vanish and $A(v)$ collapses to its remaining $0.05 \cdot w(v)$ QoS term: $A_{\max}$ sits at exactly 0.0500 (Enterprise, AV, Microservices, Hub-and-Spoke) or within rounding (ATM 0.0492, Financial 0.0455), with $\sigma_A \in [0.0145, 0.0398]$ against $\sigma_{FT} \in [0.147, 0.171]$. The rule-based SPOF detector, reading the same membership, corroborates this: mean $F_1 = 0.0042$, zero predicted SPOFs in seven of eight scenarios.

This is not evidence that single points of failure are absent, only that a hard cut-vertex test cannot see them in a topology this redundant — on IoT the detector reaches precision 1.0 at recall 0.017, so the one component it flags is genuinely failure-critical and it is blind to the rest. A redundantly but load-bearingly connected component, such as the sole active publisher into a topic with structurally-possible-but-unused alternate routes, fragments nothing when removed yet measurably degrades reachability.

We therefore replaced CDI's cut-vertex gate with a continuous redundancy-deficit measure: for every node in the main connected component, the fractional increase in average shortest-path length from a fixed sample

of high-degree sources when v is removed, capped at 1.0 on genuine fragmentation — so a true cut vertex stays at the ceiling (the prior semantics are the limiting case) while a redundant-but-load-bearing node registers a graded score. We rebalanced $A(v)$'s AHP weights accordingly, promoting CDI from $a_{CDI} = 0.10$ to parity with the hard term ($a_{CDI} = a_{AP_c} = 0.2563$) since both now measure removal-impact severity at different graduations, and shrinking a_{QSPOF} from 0.25 to 0.1998 to bound its overlap with AP_c. Recomputing moves σ_A to $[0.0254, 0.0530]$, roughly 1.6–2.0× its prior range, and $A_{\max}$ off the ceiling everywhere (Enterprise: $0.0500 \to 0.0878$). $A(v)$ is no longer a rescaled QoS weight with zero structural content, though it correctly remains narrower-spread than $FT(v)$: genuine fragility should be the exception.

7.3.9. Anti-Pattern Detection and CI/CD Quality Gates

Validating our rule-based anti-pattern catalog against the composite oracle $I_{\text{comp}}(v)$ yielded mean detection $F_1 = 0.3781$ across 8 scenarios — but F_1 understates what the catalog does: mean precision is 0.239 against mean recall 0.900, so it flags 94.2% of components on average, trading precision for near-exhaustive coverage of true positives. This is a deliberate high-recall CI/CD posture (a missed critical component costs more than a false positive requiring manual triage), and should be read as such rather than via F_1 alone. One detector, `DEEP_PIPELINE`, is excluded: it enumerates every simple source-to-sink path and does not terminate within ten minutes on the 50-application Healthcare topology — itself a reportable scalability limit of exhaustive path enumeration. The remaining 18 detectors ran in 0.04 s (50 nodes) to 54.85 s (300 nodes), inside standard pre-commit and pull-request budgets.

Stratified against pooled. We stratify the RM composite's rank correlation by node type: $\rho = 0.503$ (Application), 0.395 (Broker), 0.142 (Node), while the *pooled* correlation across all types is $\rho = 0.028$ — outside the per-type range entirely. This is a Simpson's-paradox effect: aggregating across populations with different scales and base rates reverses the within-group trend, so the pooled figure is not a summary of the per-type ones, and only the stratified numbers should be quoted when discussing RM's structural predictive power.

7.3.10. Using RM Correctly: Score Within a Node Type

The stratification above is not only a reporting convention; it is a usage rule, and getting it wrong changes which components an architect is told to fix.

The rule. Score, tier and rank components *within* a node type. Never derive a single ranking or threshold from a population mixing Applications, Brokers, Topics, Infrastructure Nodes and Libraries: their scores differ in scale and base rate, so a shared threshold measures type membership as much as risk. The layer projections of Section 3.3 are the stratification device (π_{infra}, π_{mw}, π_{app}), with only π_{system} spanning all five types; read that one as a structural and visualisation view, and take gating decisions from the single-type layers.

What pooling costs, measured. Classifying every component in the eight-scenario corpus twice — once against one box-plot over the whole system layer, once within its node type — changes the tier of **62.8%** of 1,619 components, and **19.0%** cross the CRITICAL/HIGH boundary that determines whether a component is surfaced at all. The effect is stable across scenarios (tier changes 51–69%, boundary crossings 14–22%), so it is a property of mixing populations rather than of any topology, and its direction is systematic: pooling suppresses the top of whichever type carries the lower score scale. The classifier now derives quartiles and fences within each node type, with types too small for stable quartiles falling back to the pooled fence. The residual case is π_{app}, which still pools Applications with Libraries; their LOSO correlations overlap (0.195 against 0.105), so we retain the pairing as a scope condition.

7.3.11. HGT Attention Weight Analysis

Extracting relational attention matrices from the trained HGT model on the ATM Case Study (Figure 5) revealed that the model places its highest single attention weight ($\alpha_{uv} = 1.00$) on a `USES` edge (an Application's dependency on a shared Library), with the next tier ($\alpha_{uv} \approx 0.50$) split between a `ROUTES` edge (Broker → Topic) and further `USES`/`SUBSCRIBES_TO` edges. This pattern — dominant attention on library-dependency and broker-routing channels rather than direct publish/subscribe message flow — reflects the shared-library

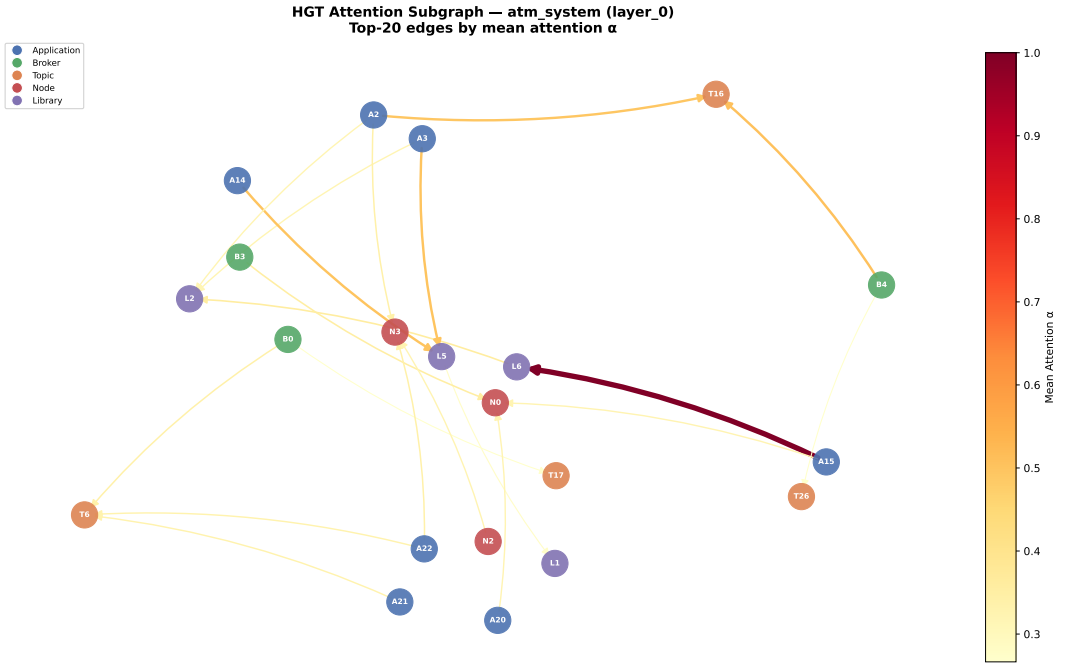

Figure 5: Relational attention extracted from the trained HGT on the ATM case study. The highest-weighted channels are `USES` (Application → shared Library) and `ROUTES` (Broker → Topic) rather than direct publish/subscribe edges, matching the shared-library blast-radius and broker-centrality pathways found by the edge-removal analysis.

blast-radius and broker-centrality failure pathways identified elsewhere in this study (Section 7.2's edge-removal analysis, Section 3's simultaneous-failure semantics for shared libraries) rather than sequential pub-sub cascade propagation.

7.4. RQ4: Real-World Distributed Software Architecture Validation

To assess external validity on authentic production architectures, we evaluated SaG on three open-source distributed systems.

Table 15: Empirical validation on authentic real-world distributed software architectures.

Real-World Architecture	$\|V\|$	$\|V_{\text{app}}\|$	Spearman ρ	Kendall τ	$F_1@K$	Tie-Robust F_1	Non-Zero	Gain
Autoware.universe (ROS 2)	75	32	**0.688 ± 0.009**	0.517	**0.800**	0.800	19 / 32	+0.360
Cloud Microservices Mesh	60	22	**0.778 ± 0.001**	0.639	**1.000**	0.760	8 / 22	+0.014
Train-Ticket Booking Mesh	90	41	**0.759 ± 0.001**	0.605	**1.000**	0.810	14 / 41	+0.264

Key Insights for RQ4:

1. **Strong Real-World Rank Agreement:** SaG achieves high rank correlation on Cloud Microservices ($\rho = 0.778$) and Train-Ticket ($\rho = 0.759$), and solid agreement on Autoware.universe ($\rho = 0.688$).
2. **Critical-Set Containment:** Every single application with non-zero cascading impact in Cloud Microservices and Train-Ticket is successfully captured within the predicted top-K critical set (tie-robust $F_1@K = 0.760$ and 0.810).
3. **Substantial Predictive Gain:** SaG outperforms raw degree centrality by +0.360 on Autoware and +0.264 on Train-Ticket, demonstrating that typed dependency derivation captures critical architectural semantics beyond superficial connectivity.

7.5. RQ5: Analysis Cost, Computational Sustainability, and CI/CD Feasibility

RQ5 asks what the analysis costs at CI/CD time, which stage dominates that computational footprint, and how it impacts sustainable software engineering. A learned model invites the question of whether it remains usable as systems grow; on this pipeline the answer is that the neural network is not the constraint — the deterministic structural analysis that feeds it is, by more than two orders of magnitude.

Table 16: Per-stage cost of the deployed (inference) path on generated systems of increasing size, CPU, median of three runs with a warm-up pass excluded. Training is not included: it is a one-off cost paid once over the corpus, not per analysed system.

$\|V\|$	$\|E\|$	**Analyse (s)**	**Graph→tensor (s)**	**HGT forward (ms)**	**Analyse : forward**
249	1,127	0.27	0.011	22.5	12×
499	2,402	0.95	0.022	15.7	61×
999	6,422	4.74	0.055	36.7	129×
1,998	19,301	23.83	0.153	43.7	**545**×

The GNN is the cheapest and greenest stage. Inference on a 2,000-component system takes 43.7 ms — a handful of sparse matrix products whose cost grows close to linearly in $|E|$ — while the structural analysis preceding it takes 23.8 s. The ratio widens with scale rather than narrowing, from 12× at 249 components to 545× at 1,998. Any effort spent making this framework scale should therefore go to the classical graph metrics, not to the model: betweenness is $O(|V||E|)$ and the directed articulation score and closeness are $O(|V|(|V|+|E|))$, giving an overall $O(|V|^2+|V||E|)$. One mitigation already ships, with the connectivity-degradation term switching to deterministic top-50 core sampling above $|V| = 300$.

Computational Sustainability and Energy Savings. From an environmental and computational sustainability perspective, replacing multi-seed discrete-event simulation sweeps with an HGT forward pass fundamentally curtails CI/CD energy consumption. Performing 5-seed dynamic fault injection and message-flow simulation across a 2,000-component topology requires several minutes to hours of compute across cluster nodes, consuming tens to hundreds of kilojoules (50–200 kJ) per commit verification. In contrast, the 43.7 ms forward pass requires negligible energy ($\ll 1$ J on a standard workstation CPU), achieving a $> 99.9\%$ reduction in evaluation energy. This speed and efficiency make continuous, per-commit architectural verification environmentally sustainable and operationally viable without inflating cloud bills.

Cost tracks edges, not components. The corpus scenarios make this concrete: the 520-component Enterprise mesh takes 27.2 s to analyse while a *denser-in-name-only* 999-component generated system takes 4.7 s, because the former carries 3,245 structural edges against a far sparser fan-out per component. Practitioners sizing a CI/CD budget should estimate from $|E|$, or from $|V| \cdot |E|$, rather than from component count. Within the paper's corpus the whole gate — analysis plus all 18 evaluated detectors — runs in 0.02 s (29 components) to 27.4 s (520 components), comfortably inside a pull-request budget.

What we have not measured, and do not claim. Nothing above 2,000 components has been timed, and nothing here should be extrapolated past it; all figures are single-threaded CPU, with no GPU measurement; and one detector, `DEEP_PIPELINE`, is excluded throughout because exhaustive source-to-sink path enumeration does not terminate within ten minutes even at 50 applications (Section 7.3). That detector is the one component of the framework known *not* to scale, and its exclusion is a limitation rather than a tuning choice. Training cost, for completeness: the full seven-variant, eight-fold, five-seed leave-one-scenario-out sweep reported in Table 9 took approximately 45 minutes per learned variant on CPU.

8. Discussion, Threats to Validity, and Conclusion

8.1. Discussion and Practical Implications

Our empirical findings provide clear guidance for software architects and reliability engineers:

- **What the Predictive Pathway Is For:** The typed model earns its place on three counts. It is the best configuration for evaluating an architecture never analysed before ($\rho = 0.608$ out of distribution), where the untyped alternative fails to transfer at all ($\rho = 0.086$). It is the strongest identifier of the

critical top-K shortlist hardening actually consumes ($F_1@K = 0.414$). And it answers in 44 ms, letting architectural risk be gated on every pull request rather than swept nightly (Section 7.5). Beyond the ranking it supplies typed attention over the relations that carried a cascade, per-relationship criticality, and multi-task quality outputs — none of which a centrality score produces.

- **Where the Boundary Is — and When Not To Train:** The honest comparison is against a training-free QoS-weighted centrality baseline, and there the ranking margin is +0.037 and not statistically significant ($p = 0.64$, Table 9). A team that needs a critical-component ranking and *nothing else*, on architectures resembling ones it has already characterised, should use Topo-QoS: it costs no training, no corpus, and no checkpoint. The learned model earns its cost when the additional outputs above are wanted, or when the deployment target differs structurally from anything in the training corpus. Between the two typed variants: HGL for in-distribution accuracy, HGL-QoS otherwise (Section 7.3).

- **What the Explanation Layer Adds:** The deterministic RM profile is what makes a ranked set actionable. Whether a service is critical through single-point-of-failure exposure (Availability) or wide error propagation (Fault Tolerance) dictates whether to add load-balanced replicas or circuit-breaker policies — a distinction neither the predictor nor the oracle expresses.

This split mirrors Figure 1: the explanation layer (RM/$Q(v)$) and the predictive pathway (Topo-BL/Topo-QoS/GL/HGL) answer different questions from the same graph, so Table 9 and Figure 3 report RM alongside the ranking predictors as a reference point rather than as a competing entry in that comparison.

8.2. Performance and Computational Sustainability Implications

The cost profile of Section 7.5 inverts the conventional expectation of deep learning pipelines: the learned model is by far the *cheapest and greenest* stage, and the efficiency advantage widens monotonically with system scale (43.7 ms inference against 23.8 s of structural feature extraction at 2,000 components; 12× at 249 nodes, 545× at 1,998 nodes). Two critical software engineering consequences follow:

1. Green CI/CD Quality Gating vs. Simulation Energy Waste. In modern cloud-native continuous integration pipelines, evaluating architectural resilience via continuous chaos engineering or multi-seed discrete-event simulation is computationally prohibitive. Performing a 5-seed dynamic fault injection and message-flow simulation sweep across a 2,000-component topology requires minutes to hours of distributed CPU cluster time, dissipating an estimated 50–200 kJ of energy per commit build. In contrast, evaluating our relation-specific HGT forward pass requires only 43.7 ms on a standard workstation CPU ($\ll 1$ J), achieving a $> 99.9\%$ reduction in verification energy footprint. This enables continuous, per-commit architectural verification without inflating cloud operational expenditure or CI/CD carbon emissions.

2. Operational Sustainability via Avoided Cascade Energy. Beyond pre-deployment testing budgets, the primary sustainability dividend of SaG lies in *avoided operational compute* in production environments. When an architectural single-point-of-failure or unbuffered dependency collapses under load, the resulting systemic cascade triggers runaway retry storms, exponential backoff polling, container restart thrashing, and database connection pool starvation. We can formalize the avoided operational cascade energy $\Delta E_{\text{cascade}}$ across the impacted component set V_{cascade} as:

$$\Delta E_{\text{cascade}} = \sum_{v \in V_{\text{cascade}}} \left[E_{\text{restart}}(v) + E_{\text{retry}}(v) + E_{\text{tail}}(v) \right], \tag{21}$$

where $E_{\text{restart}}(v)$ represents the CPU and memory provisioning energy expended in repeatedly re-initializing crashed services, $E_{\text{retry}}(v)$ models the network and compute overhead of unthrottled downstream retransmissions, and $E_{\text{tail}}(v)$ accounts for the active CPU cycles consumed while worker threads wait in saturated queue backpressure. In large-scale cloud services, a single cascading outage lasting tens of minutes dissipates hundreds of kilowatt-hours (kWh) of unavailing electrical energy. By detecting and mitigating these failure paths at design time within pull-request gates, SaG eliminates this operational energy sink before code reaches staging or production.

Boundary and Explicit Threat. We state its empirical boundary plainly: **we performed no direct physical hardware power measurement in this study**. All reporting is single-threaded CPU wall-clock execution time; we did not instrument package energy via Intel RAPL or GPU-side power via NVIDIA NVML, nor have we quantified the exact energy of physical production failures. The operational sustainability claim is therefore structurally reasoned and conditional on incident prevention rather than measured in joules on physical power meters. Direct power instrumentation is scheduled for future work (Section 8.4).

8.3. Threats to Validity

- **Construct Validity:** Our ground truth is generated via discrete-event cascade simulation on structural models rather than observing live production outages. To characterise construct divergence we evaluated the three reliability-facing simulation engines (`FaultInjector`, `FailureSimulator`, `MessageFlowSimulator`) of our four-oracle taxonomy (Section 4.3). On *ordering* the evidence is genuinely convergent: the behavioural queue-flow simulator and the topological cascade oracle agree at $\rho = 0.883$, never below 0.756 on any scenario, despite being built on different principles — though that convergence is about *service disruption reach*, not about the operational criticality of the messages lost: the queue-flow simulator's label is an unweighted delivery-rate delta, blind to message priority and only partly sensitive to reliability policy. On *critical-set membership* it is not: top-K Jaccard is 0.31–0.42 across pairs against 0.111 expected by chance, and this is neither a tie-breaking artifact (≤ 0.005 change under a tie-robust cut) nor purely construct divergence — the labeler compared against its own reruns reaches only 0.44 in its worst scenario, so the statistic is unstable at this K for any oracle. We also could not close the gap by weighting: applying the same $w(t)$ to both topological oracles changes their agreement by $\Delta\rho = -0.009$, a null result we report rather than omit. A further 30–47% of components per scenario carry no simulated ground truth at all, because the cascade model cannot express direct Topic or Node failure. Accordingly, results established against one oracle should not be read as claims about another, and none of them are claims about observed production outages: the reported correlations measure surrogate fidelity to a simulator, not predictive accuracy against deployed systems. The fourth oracle, $I_M(v)$ (`ChangePropagationSimulator`), stands outside this comparison because it targets Maintainability rather than a competing reliability ranking, and it carries a construct-validity limitation of its own worth stating plainly: $M(v)$ (Section 5.2) carries $q_M = 0.20$ of the composite $Q(v)$, yet this paper reports no maintainability correlation, gate, or table against it. The reason is not merely that the labeler never observed it (above) — $I_M(v)$ *is* computed on every Validate-stage sweep — but that $I_M(v)$ traverses the same derived dependency topology $M(v)$ is scored from, so agreement between them would be an internal consistency check rather than independent evidence. Settling this would require an external referent uncorrelated with that topology, such as code churn or co-change coupling mined from commit history; we did not measure this for our three real-world architectures and leave it to future work.

- **Internal Validity:** Potential feature leakage is prevented by strict graph view separation: predictors operate on G_{analysis}, whereas ground-truth simulators operate on $G_{\text{structural}}$. We additionally identified and fixed a normalization defect in the code-quality scoring pipeline: a min-max helper previously assigned the maximal Code Quality Penalty to any zero-variance population, which is correct for one genuine node but was indistinguishable from "many nodes carrying no code-quality data at all" — the exact shape of every Library in our three real-world scenarios, which carry no source-level `code_metrics`. The fix (scoring an undifferentiated multi-node population as zero rather than maximal penalty) changes Library Maintainability tier classifications in the real-world corpus but, as expected for a fix confined to a population with no variance to rank, leaves Application-population rank correlation against $I^*(v)$ effectively unchanged (Table 15: $\Delta\rho \in \{-0.003, 0.000, 0.000\}$ across the three real-world scenarios). A second, unresolved confound belongs here too: per-topic QoS in four of our seven synthetic scenarios is generated by a domain-keyed lookup whose table collapses to a single entry, so $w(t)$ has standard deviation ≈ 0.02 there against 0.19–0.28 in the other three (Section 6.1). `Topo-QoS` nonetheless gains a mean $+0.305\,\rho$ over `Topo-BL` on exactly those four scenarios (AV, Enterprise, Hub-and-Spoke, Microservices; Table 7), a gain that cannot be QoS discrimination given the absent

signal, and must instead originate in the weighting scheme's other effects — for instance its uniform rescaling of pub/sub-derived `DEPENDS_ON` edge weights relative to `USES`-derived ones. We did not run the diagnostic that would separate the two (re-scoring `Topo-QoS` with $w(t)$ forced constant), so we report the confound rather than an explanation for it; the same corpus property is the more likely driver of Section 7.3's $\Delta\rho = -0.009$ QoS-convergence null than the ladder clipping we attribute it to there.

- **External Validity:** While our corpus spans ten architectures across six declared system domains, three of them authentic open-source systems (ROS 2 and microservices), future work should evaluate larger enterprise deployments with hundreds of microservices.

- **Sustainability Claims Are Unmeasured:** The efficiency argument in Section 8.2 rests on wall-clock cost and on avoided recovery compute, neither of which we instrumented for energy. No power, energy or carbon figure appears in this paper, and none should be inferred from the timing tables; a joules-per-analysis claim, or a claim about energy saved per prevented outage, would require hardware counters and a live incident testbed we did not have.

- **Conclusion Validity:** Given the heavy-tailed, non-normal distribution of cascading failure impacts, all statistical comparisons utilize non-parametric rank correlation (Spearman ρ, Kendall τ), bootstrap confidence intervals ($B = 2,000$), and paired Wilcoxon signed-rank tests. Two hazards deserve explicit statement because both changed conclusions in this study rather than merely threatening to.

  First, *aggregation across node types.* The RM composite's rank correlation pooled across types ($\rho = 0.028$) falls outside the range spanned by its own per-type correlations ($\rho \in [0.14, 0.50]$) — a Simpson's-paradox effect. Reported pooled, the same LOSO experiment placed the RM baseline at $\rho = -0.014$ and the untyped GL model at 0.381; reported on the Application population it places them at $+0.195$ and 0.086 respectively, reversing their order. Every figure in this paper is therefore scored on a single stated population (Section 6.3), and we quote no pooled cross-type correlation.

  Second, *substrate.* An earlier version of the RM ablations in Section 7.3 scored $Q(v)$ from features restricted to the Application–Library `DEPENDS_ON` projection — the substrate built for GNN feature/label alignment — on which no Application is an articulation point and no incident edge is a bridge in six of eight scenarios. Four of Availability's five terms vanish there, leaving $A(v)$ (roughly 0.51 of the composite) constant, and all three affected sweeps consequently reported negative ρ for a baseline that is in fact weakly positive. The ablations are now computed through the full analysis pipeline. We record this because the failure mode is not visible in the output: a degenerate score produces plausible-looking correlations, and only checking the variance of each term exposed it.

8.4. Limitations and Future Work

1. **Distributed AI and LLM Serving Topologies:** Modern AI infrastructure relies on distributed LLM serving systems (e.g., vLLM, Triton, DeepSpeed) characterized by complex tensor and pipeline parallelism across GPU nodes, dynamic KV-cache routing, and disaggregated prefill-decode disaggregation. A failure or straggler node in a pipeline-parallel ring induces severe head-of-line blocking and massive GPU idle power dissipation. Extending the SaG multigraph schema to model distributed model serving topologies (nodes representing GPU workers, edges encoding tensor communication channels and KV-cache transfer fabrics) offers a promising avenue to predict and mitigate bottlenecks in AI serving clusters.
2. **Physical Hardware Power Counter Instrumentation:** Validating the sustainability thesis through empirical power measurements using running package counters (Intel RAPL, AMD RAPL, NVIDIA NVML) and IPMI baseboard sensors on physical Kubernetes clusters during live fault-injection and chaos experiments.
3. **Hardware-in-the-Loop (HIL) Cyber-Physical Validation:** Validating SaG's predictions against real-time physical fault-injection testbeds in cyber-physical environments, such as autonomous driving compute boxes running ROS 2 with CAN-bus hardware interfaces.

4. **Automated Architectural Refactoring and Self-Healing:** Extending SaG from predictive analysis to prescriptive synthesis—automatically generating pull requests that reconfigure QoS policies, insert circuit-breakers, and add redundant broker pathways to eliminate single points of failure.

8.5. Conclusion

We presented **Software-as-a-Graph (SaG)**, a pre-deployment Static System Analysis framework that bridges the Architecture–Code Gap by combining a relation-specific Heterogeneous Graph Transformer (HGT) for failure-impact forecasting with an interpretable ISO/IEC 25010 Reliability–Maintainability explanation layer. SaG addresses the core trifecta of modern software engineering: **performance** (providing sub-second 43.7 ms pull-request gating and pinpointing queue saturation bottlenecks), **reliability** (achieving inductive cross-topology failure blast-radius forecasting across unseen architectures), and **computational sustainability** (replacing energy-intensive multi-seed chaos simulations with a low-power AI surrogate that eliminates operational cascade energy waste).

Our empirical results across synthetic systems and authentic open-source distributed platforms (ROS 2 Autoware, Cloud Microservices, Train-Ticket) demonstrate that relation-specific typing is what enables graph learning to generalize out of distribution: the typed HGT model reaches $\rho = 0.608$ out of distribution where untyped models collapse to $\rho = 0.086$ ($p = 0.0078$). Simultaneously, SaG captures the critical top-20% components with $F_1@K = 0.414$, while its ISO-grounded explanation layer opens the AI black box by decomposing risks into concrete Availability, Fault Tolerance, and Maintainability drivers. By evaluating architectures in 43.7 ms before code is deployed, SaG delivers a transparent, performant, and sustainable foundation for engineering dependable distributed software systems.

CRediT authorship contribution statement

[Omitted for double-anonymised review. To be completed on acceptance with the standard CRediT roles: Conceptualization; Methodology; Software; Validation; Formal analysis; Investigation; Data curation; Writing – original draft; Writing – review and editing; Visualization; Supervision.]

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Funding

[Omitted for double-anonymised review.]

Data availability

The replication package—including the seven synthetic scenario datasets and the configurations that generate them, the topology generator, the four simulation harnesses, the real-world architecture adapters, and every analysis script behind the reported tables and figures—will be made openly available upon publication. The synthetic corpus is regenerable rather than merely archived: each dataset carries its seed and a SHA-256 digest in a committed manifest, and a regression test asserts byte-identical regeneration from the configurations (Section 6.1.1). Every table and figure in this paper is produced from a committed artifact by a script in that package; none of the reported values is transcribed by hand.

Declaration of generative AI use

[To be completed by the authors in accordance with the journal's policy.]

References

[1] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, W. Woodall, Robot operating system 2: Design, architecture, and uses in the wild, Science Robotics 7 (66) (2022) eabm6074.

[2] J. Kreps, N. Narkhede, J. Rao, Kafka: A distributed messaging system for log processing, in: Proc. 6th Int. Workshop on Networking Meets Databases (NetDB), 2011.

[3] Object Management Group, Data distribution service (dds), Tech. Rep. formal/2015-04-10, version 1.4, Object Management Group (2015).

[4] OASIS, MQTT version 5.0, OASIS Standard (2019).

[5] P. T. Eugster, P. A. Felber, R. Guerraoui, A.-M. Kermarrec, The many faces of publish/subscribe, ACM Computing Surveys 35 (2) (2003) 114–131.

[6] International Organization for Standardization, ISO/IEC 25010:2023 — systems and software engineering — systems and software quality requirements and evaluation (square) — product quality model, Tech. rep., International Organization for Standardization (2023).

[7] International Organization for Standardization, ISO/IEC 25019:2023 — systems and software engineering — systems and software quality requirements and evaluation (square) — quality-in-use model, Tech. rep., International Organization for Standardization (2023).

[8] T. J. McCabe, A complexity measure, IEEE Transactions on Software Engineering SE-2 (4) (1976) 308–320.

[9] S. R. Chidamber, C. F. Kemerer, A metrics suite for object oriented design, IEEE Transactions on Software Engineering 20 (6) (1994) 476–493.

[10] N. Fenton, J. Bieman, Software Metrics: A Rigorous and Practical Approach, 3rd Edition, CRC Press, 2014.

[11] A. Basiri, N. Behnam, R. de Rooij, L. Hochstein, L. Kosewski, J. Reynolds, C. Rosenthal, Chaos engineering, IEEE Software 33 (3) (2016) 35–41.

[12] L. C. Freeman, A set of measures of centrality based on betweenness, Sociometry 40 (1) (1977) 35–41.

[13] S. Brin, L. Page, The anatomy of a large-scale hypertextual web search engine, Computer Networks and ISDN Systems 30 (1–7) (1998) 107–117.

[14] U. Brandes, A faster algorithm for betweenness centrality, Journal of Mathematical Sociology 25 (2) (2001) 163–177.

[15] M. E. J. Newman, Networks: An Introduction, Oxford University Press, 2010.

[16] [Anon-A], Authors' prior work on multi-layer graph dependency analysis for publish–subscribe systems, citation withheld for double-anonymised review (n.d.).

[17] V. R. Basili, L. C. Briand, W. L. Melo, A validation of object-oriented design metrics as quality indicators, IEEE Transactions on Software Engineering 22 (10) (1996) 751–761.

[18] N. Nagappan, T. Ball, Static analysis tools as early indicators of pre-release defect density, in: Proc. 27th Int. Conf. on Software Engineering (ICSE), 2005, pp. 580–586.

[19] T. Zimmermann, R. Premraj, A. Zeller, Predicting defects for Eclipse, in: Proc. 3rd Int. Workshop on Predictor Models in Software Engineering (PROMISE), 2007.

[20] T. Menzies, J. Greenwald, A. Frank, Data mining static code attributes to learn defect predictors, IEEE Transactions on Software Engineering 33 (1) (2007) 2–13.

[21] J. Garcia, D. Popescu, G. Edwards, N. Medvidovic, Toward a catalogue of architectural bad smells, in: Proc. 5th Int. Conf. on the Quality of Software Architectures (QoSA), LNCS 5581, 2009, pp. 146–162.

[22] D. Taibi, V. Lenarduzzi, On the definition of microservice bad smells, IEEE Software 35 (3) (2018) 56–62.

[23] Z. Li, P. Avgeriou, P. Liang, A systematic mapping study on technical debt and its management, Journal of Systems and Software 101 (2015) 193–220.

[24] J. Humble, D. Farley, Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation, Addison-Wesley, 2010.

[25] L. Chen, Continuous delivery: Huge benefits, but challenges too, IEEE Software 32 (2) (2015) 50–54.

[26] International Organization for Standardization, ISO/IEC 25023:2016 — systems and software engineering — systems and software quality requirements and evaluation (square) — measurement of system and software product quality, Tech. rep., International Organization for Standardization (2016).

[27] International Organization for Standardization, ISO/IEC 25021:2012 — systems and software engineering — systems and software quality requirements and evaluation (square) — quality measure elements, Tech. rep., International Organization for Standardization (2012).

[28] T. L. Saaty, The Analytic Hierarchy Process: Planning, Priority Setting, Resource Allocation, McGraw-Hill, 1980.

[29] R. Albert, H. Jeong, A.-L. Barabási, Error and attack tolerance of complex networks, Nature 406 (2000) 378–382.

[30] A. E. Motter, Y.-C. Lai, Cascade-based attacks on complex networks, Physical Review E 66 (2002) 065102(R).

[31] S. V. Buldyrev, R. Parshani, G. Paul, H. E. Stanley, S. Havlin, Catastrophic cascade of failures in interdependent networks, Nature 464 (2010) 1025–1028.

[32] C. Fan, L. Zeng, Y. Sun, Y.-Y. Liu, Finding key players in complex networks through deep reinforcement learning, Nature Machine Intelligence 2 (2020) 317–324.

[33] C. Fan, L. Zeng, Y. Ding, M. Chen, Y. Sun, Z. Liu, Learning to identify high betweenness centrality nodes from scratch: A novel graph neural network approach, in: Proc. 28th ACM Int. Conf. on Information and Knowledge Management (CIKM), 2019, pp. 559–568.

[34] A. Varbella, K. Amara, M. El-Assady, B. Gjorgiev, G. Sansavini, PowerGraph: A power grid benchmark dataset for graph neural networks, in: Advances in Neural Information Processing Systems 37 (NeurIPS 2024), Datasets and Benchmarks Track, 2024, arXiv:2402.02827.

[35] T. N. Kipf, M. Welling, Semi-supervised classification with graph convolutional networks, in: Proc. Int. Conf. on Learning Representations (ICLR), 2017.

[36] W. L. Hamilton, R. Ying, J. Leskovec, Inductive representation learning on large graphs, in: Advances in Neural Information Processing Systems 30 (NeurIPS), 2017, pp. 1024–1034.

[37] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, Y. Bengio, Graph attention networks, in: Proc. Int. Conf. on Learning Representations (ICLR), 2018.

[38] M. Schlichtkrull, T. N. Kipf, P. Bloem, R. van den Berg, I. Titov, M. Welling, Modeling relational data with graph convolutional networks, in: Proc. European Semantic Web Conference (ESWC), 2018, pp. 593–607.

[39] X. Wang, H. Ji, C. Shi, B. Wang, Y. Ye, P. Cui, P. S. Yu, Heterogeneous graph attention network, in: Proc. The Web Conference (WWW), 2019, pp. 2022–2032.

[40] Z. Hu, Y. Dong, K. Wang, Y. Sun, Heterogeneous graph transformer, in: Proc. The Web Conference (WWW), 2020, pp. 2704–2710.

[41] X. Fu, J. Zhang, Z. Meng, I. King, MAGNN: Metapath aggregated graph neural network for heterogeneous graph embedding, in: Proc. The Web Conference (WWW), 2020, pp. 2331–2341.

[42] Z. Ying, D. Bourgeois, J. You, M. Zitnik, J. Leskovec, GNNExplainer: Generating explanations for graph neural networks, in: Advances in Neural Information Processing Systems (NeurIPS), Vol. 32, 2019, pp. 9244–9255.

[43] F. Xia, T.-Y. Liu, J. Wang, W.-S. Zhang, H. Li, Listwise approach to learning to rank: Theory and algorithm, in: Proc. 25th Int. Conf. on Machine Learning (ICML), 2008, pp. 1192–1199.

[44] Team SimPy, Simpy: Event discrete simulation for Python, `https://simpy.readthedocs.io` (2020).

[45] S. Kato, S. Tokunaga, Y. Maruyama, S. Maeda, M. Hirabayashi, Y. Kitsukawa, A. Monrroy, T. Ando, Y. Fujii, T. Azumi, Autoware on board: Enabling autonomous vehicles with embedded systems, in: Proc. ACM/IEEE 9th Int. Conf. on Cyber-Physical Systems (ICCPS), 2018, pp. 287–296.

[46] Google Cloud Platform, Online boutique: A cloud-native microservices demo application, Software artifact (2024).

[47] X. Zhou, X. Peng, T. Xie, J. Sun, C. Ji, W. Li, D. Ding, Fault analysis and debugging of microservice systems: Industrial survey, benchmark system, and empirical study, IEEE Transactions on Software Engineering 47 (2) (2021) 243–260.

[48] F. Wilcoxon, Individual comparisons by ranking methods, Biometrics Bulletin 1 (6) (1945) 80–83.

[49] B. Efron, R. J. Tibshirani, An Introduction to the Bootstrap, Chapman & Hall, 1993.

[50] C. Spearman, The proof and measurement of association between two things, American Journal of Psychology 15 (1) (1904) 72–101.

[51] A. Saltelli, M. Ratto, T. Andres, F. Campolongo, J. Cariboni, D. Gatelli, M. Saisana, S. Tarantola, Global Sensitivity Analysis: The Primer, John Wiley & Sons, 2008.

[52] M. D. Morris, Factorial sampling plans for preliminary computational experiments, Technometrics 33 (2) (1991) 161–174.

[53] F. Campolongo, J. Cariboni, A. Saltelli, An effective screening design for sensitivity analysis of large models, Environmental Modelling & Software 22 (10) (2007) 1509–1518.